Deep Learning in Artificial and Biological Neuronal Networks

Lecture #5: Reinforcement Learning.



ETH zürich

Benjamin F. Grewe
Institute of Neuroinformatics.
bgrewe@ethz.ch

University of Zurich[UZH]

Wednesday, Oct. 14th

Why is RL so important?

DeepMind › Blog › AlphaStar: Grandmaster level in StarCraft II using multi-agent reinforcement...

BLOG POST
RESEARCH

30 O

AlphaStar: Grandmaster level in StarCraft II using multi-agent reinforcement learning

DeepMind Matt Botvinick

With so much at stake, the need for the field of neuroscience and AI to come together is now more urgent than ever before.

Neuron
Volume 107, Issue 4, 19 August 2020, Pages 603-616
CellPress

Review

Deep Reinforcement Learning and Its Neuroscientific Implications

Matthew Botvinick [1, 2], Jane X. Wang [1], Will Dabney [1], Kevin J. Miller [1, 2], Zeb Kurth-Nelson [1, 2]

Show more

https://doi.org/10.1016/j.neuron.2020.06.014

Get rights and content

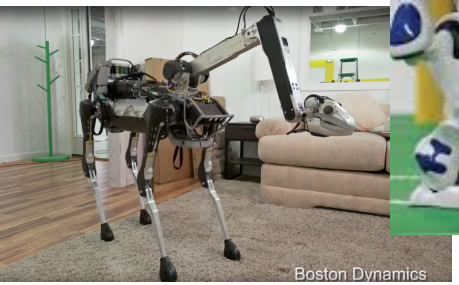
MORE VIDEOS

Why is this topic important?

Machine Learning





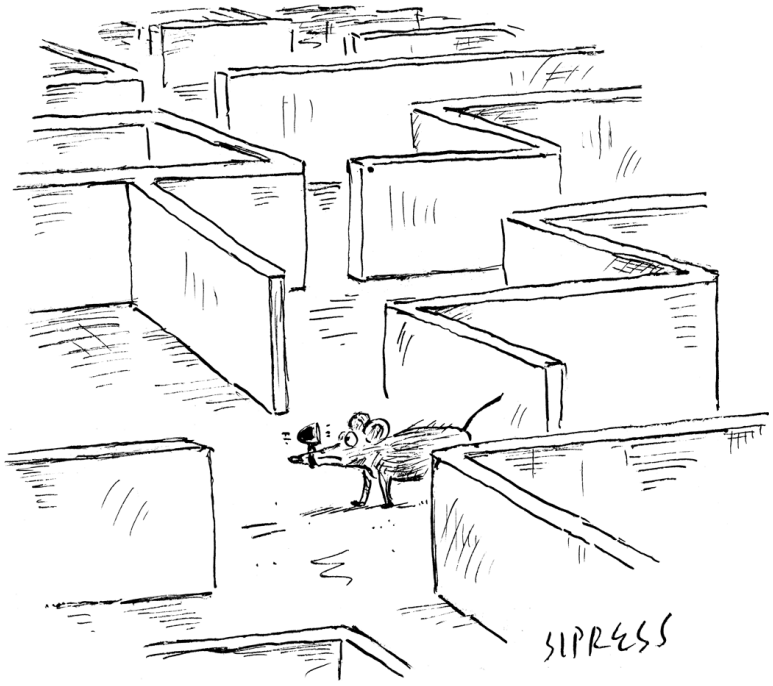






Neuroscience

Reinforcement Learning - Content Lecture #8.



"Recalculating ... recalculating ..."

What is Reinforcement Learning?

1. Introduction to Reinforcement Learning
- Reinforcement Learning and the Brain
- Example of RL Neuroscience Experiments
- The Rescorla-Wagner Rule
- The Temporal Difference Rule
- The basic Idea of Q-Learning

2. Reinforcement Learning in Machines
- Basic Components of RL
- Policy and Value Function based RL
- MC, MRPs and MDPs
- The problem of finding the optimal policy.
- The Bellmann (Expectation) Equation
- RL and Deep Q-Learning

Books and Tutorials

BOOKS:

- **An Introduction to Reinforcement Learning,**

Sutton and Barto, 1998, MIT Press
Available free online! http://incompleteideas.net/book/the-book-2nd.html

- **Algorithms for Reinforcement Learning,**

Szepesvari Morgan and Claypool, 2010
Also available for free online!

TUTORIALS:

- **RL Tutorial from David Silver (slide credit)**

http://www0.cs.ucl.ac.uk/staff/d.silver/web/Teaching.html

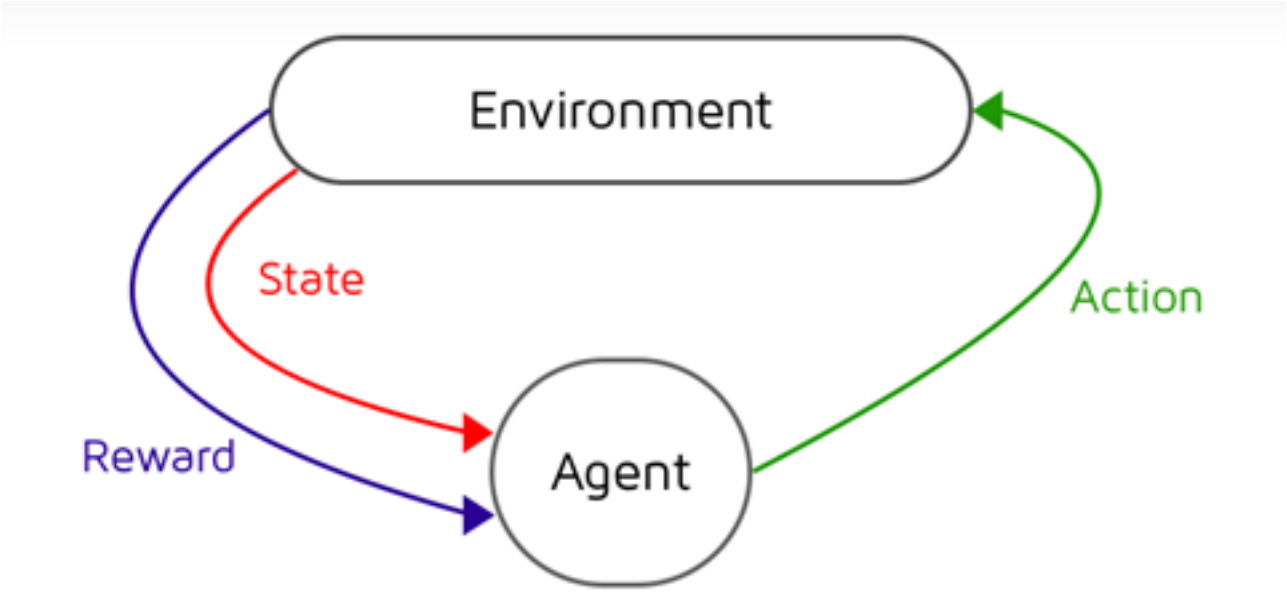
- **Tutorial on Deep RL**

https://icml.cc/2016/tutorials/deep_rl_tutorial.pdf

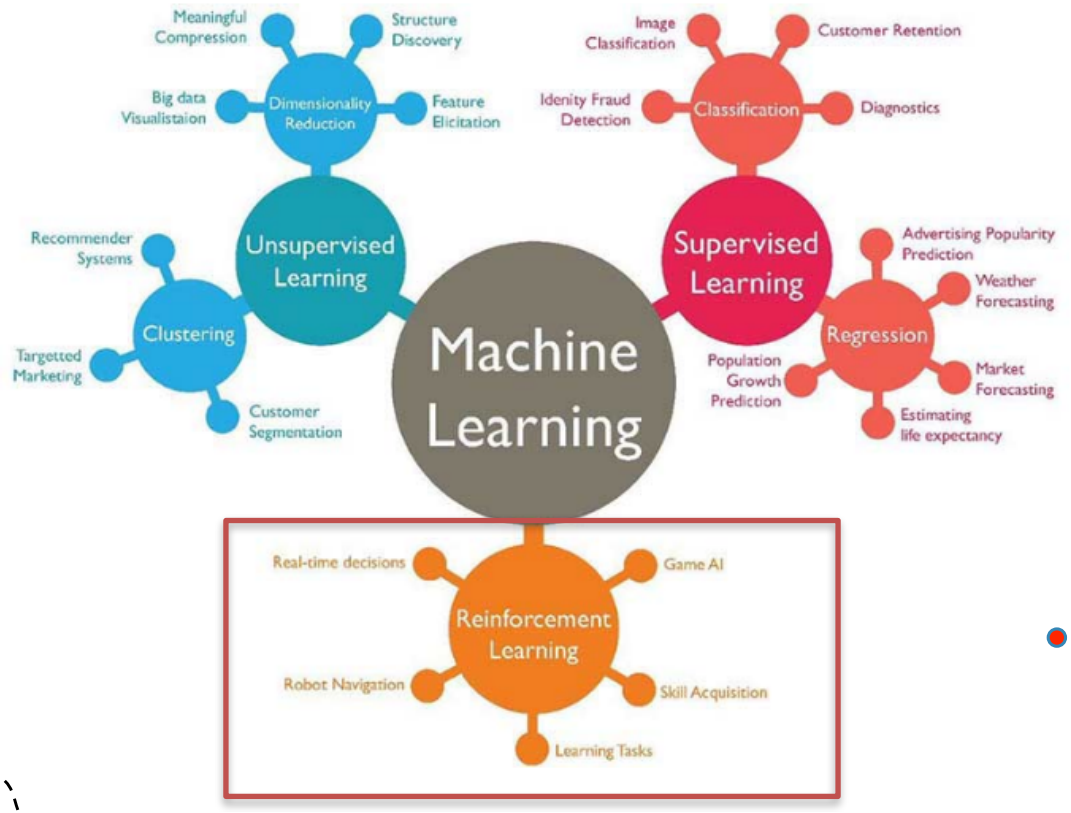
- **RL and the Brain Tutorial by Yael Niv**

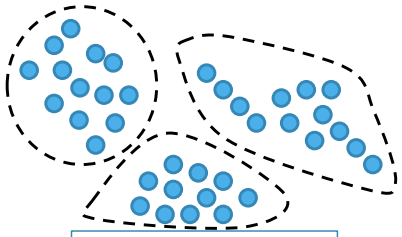
https://www.princeton.edu/~yael/Publications/Niv2009.pdf

What is Reinforcement Learning?



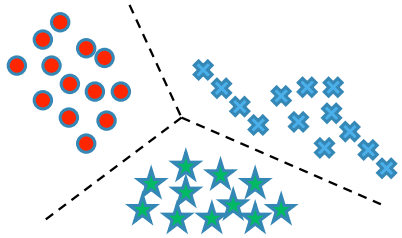
Types of Learning (ML classification)?





Unsupervised learning

Unsupervised/Self-supervised learning
Training data does not include desired outputs.



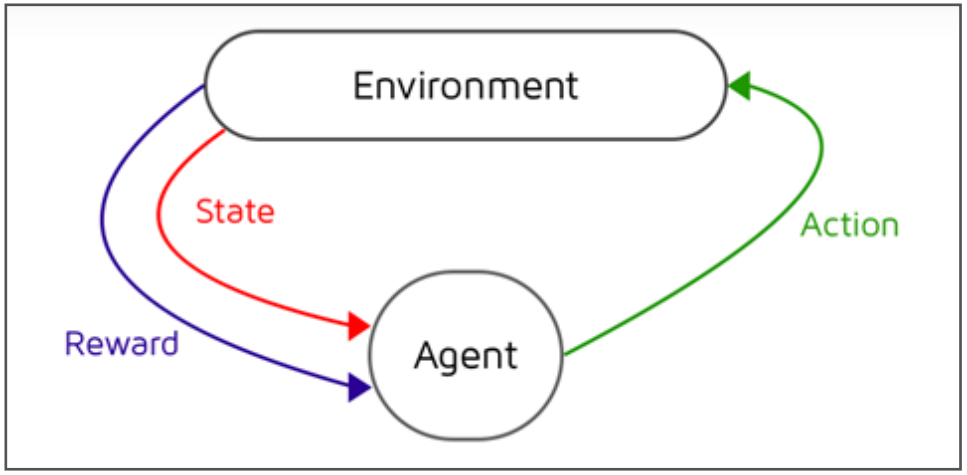
Supervised learning

Supervised learning
Training data includes desired outputs.

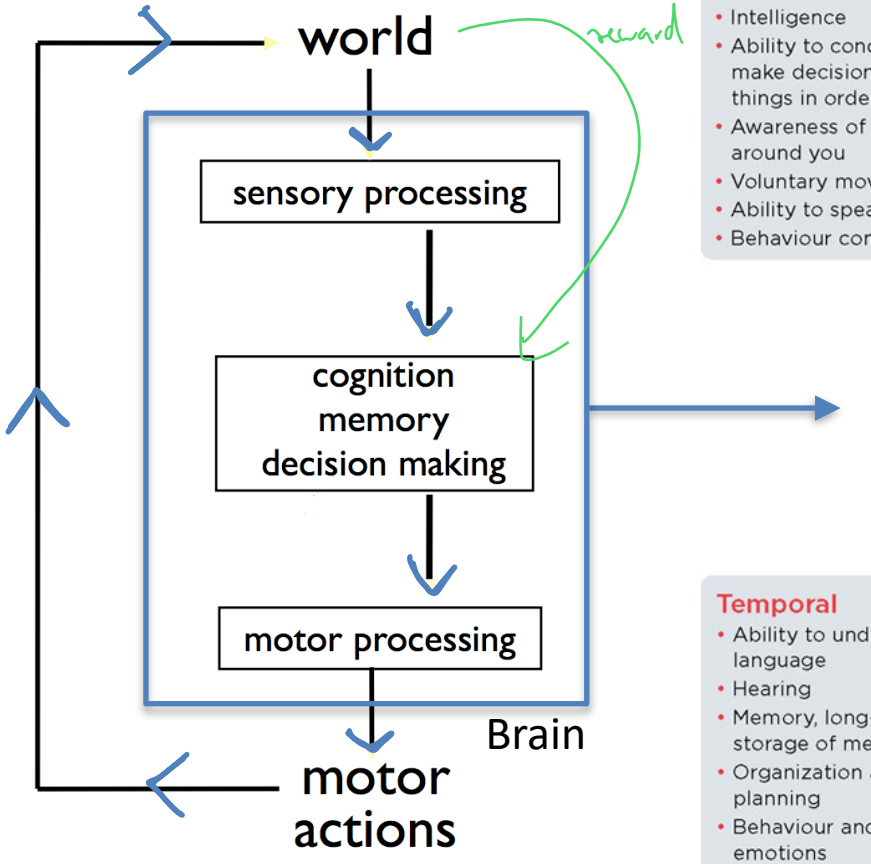
How is RL different from other types of learning?

What makes reinforcement learning different from other machine learning paradigms?

- There is only a reward/supervision signal after each action.
- Feedback is often delayed and not instantaneous.
  Time needs to be taken into account.
- The agent's actions affect the subsequent data it receives.



RL and the Brain



Frontal
- Personality
- Emotions and arousal
- Intelligence
- Ability to concentrate, make decisions, plan, put things in order, solve problems
- Awareness of what is around you
- Voluntary movement
- Ability to speak and write
- Behaviour control

Parietal
- Sensations: pain, touch, temperature
- Understanding and interpreting sensory information, such as size, colour and shape
- Understanding space and distance
- Math calculations



Occipital
- Vision
- Interpreting what you see

Temporal
- Ability to understand language
- Hearing
- Memory, long-term storage of memories
- Organization and planning
- Behaviour and emotions

Brain stem
- Breathing
- Heart rate control
- Consciousness, alertness, wakefulness
- Swallowing
- Blood pressure
- Sweating

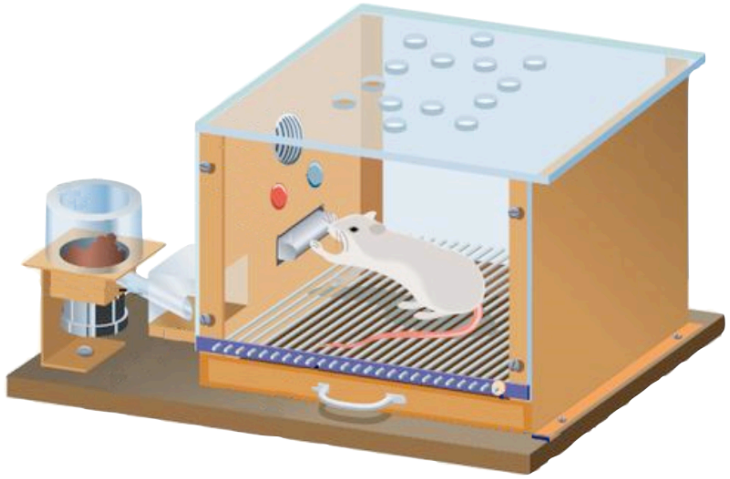
Cerebellum
- Balance
- Motor (movement) coordination
- Posture
- Fine motor skills

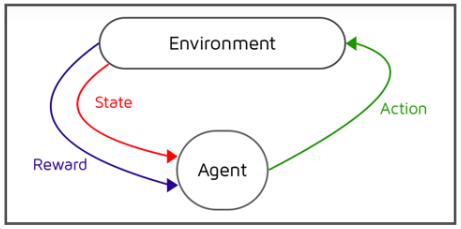
Reward Neurons in the Brain

Predictive Reward Signal of Dopamine Neurons

WOLFRAM SCHULTZ
Institute of Physiology and Program in Neuroscience, University of Fribourg, CH-1700 Fribourg, Switzerland

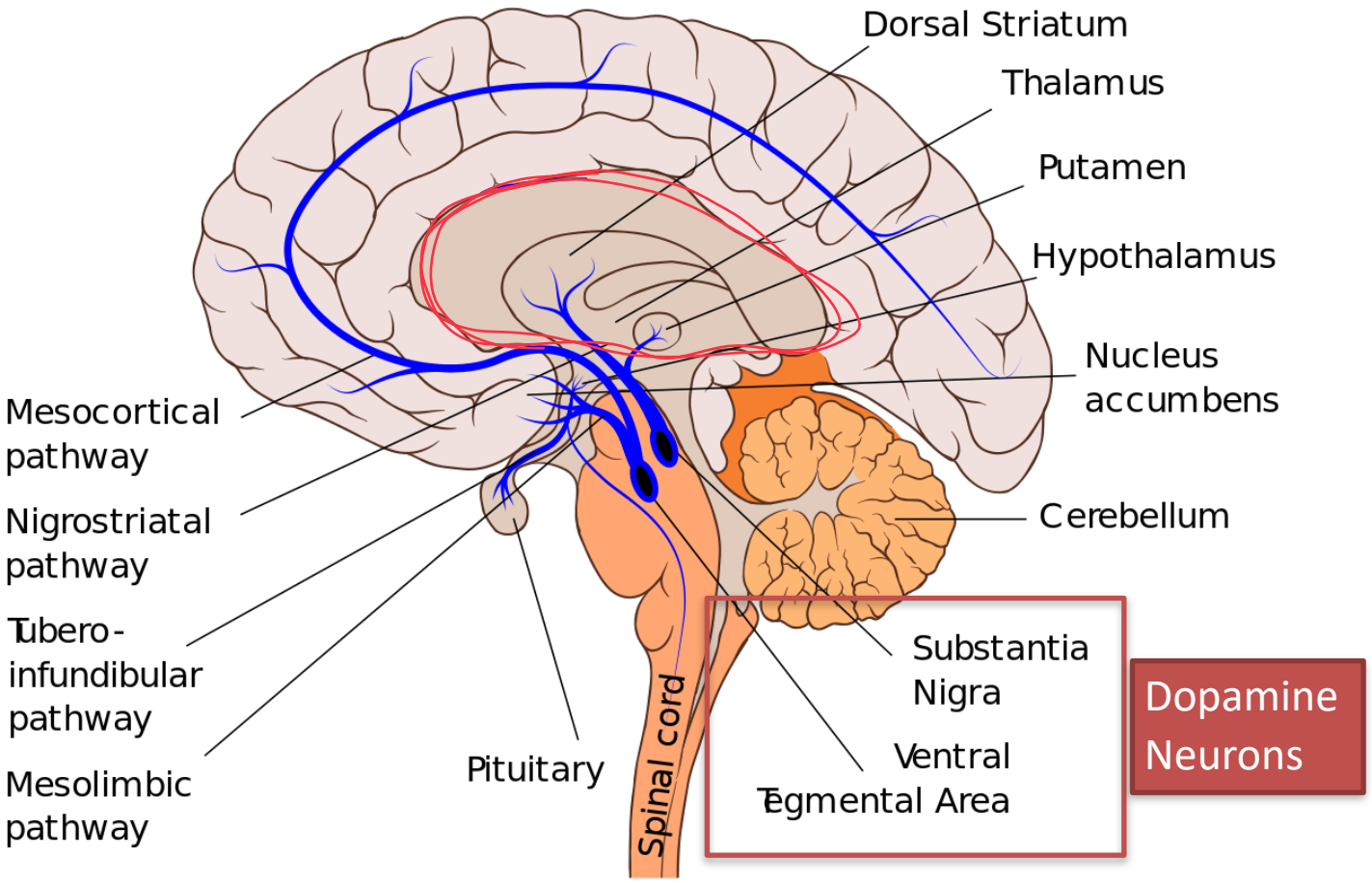




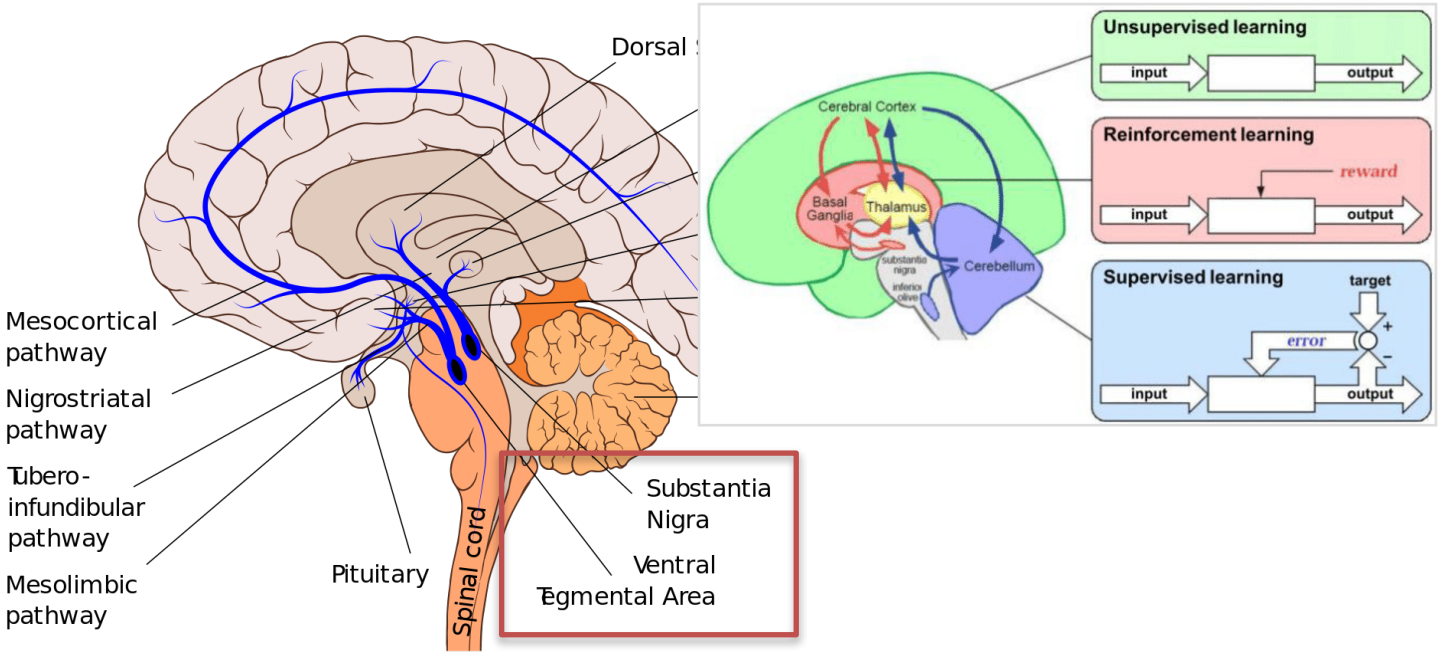


Schultz, Wolfram. Predictive reward signal of dopamine neurons. *J. Neurophysiol.* 80: 1–27, 1998. The effects of lesions, receptor blocking, electrical self-stimulation, and drugs of abuse suggest that midbrain dopamine systems are involved in processing reward information and learning approach behavior. Most dopamine neurons show phasic activations after primary liquid and food rewards and conditioned, reward-predicting visual and auditory stimuli. They show biphasic, activation-depression responses after stimuli that resemble reward-predicting stimuli or are novel or particularly salient. However, only few phasic activations follow aversive stimuli. Thus dopamine neurons label environmental stimuli with appetitive value, predict and detect rewards and signal alerting and motivating events. By failing to discriminate between different rewards, dopamine neurons appear to emit an alerting message about the surprising presence or absence of rewards. All responses to rewards and reward-predicting stimuli depend on event predictability. Dopamine neurons are activated by rewarding events that are better than predicted, remain uninfluenced by events that are as good as predicted, and are depressed by events that are worse than predicted. By signaling rewards according to a prediction error, dopamine responses have the formal characteristics of a teaching signal postulated by reinforcement learning theories. Dopamine responses transfer during learning from primary rewards to reward-predicting stimuli. This may contribute to neuronal mechanisms underlying the retrograde action of rewards, one of the main puzzles in reinforcement learning. The impulse response releases a short pulse of dopamine onto many dendrites, thus broadcasting a rather global reinforcement signal to postsynaptic neurons. This signal may improve approach behavior by providing advance reward information before the behavior occurs, and may contribute to learning by modifying synaptic transmission. The dopamine reward signal is supplemented by activity in neurons in striatum, frontal cortex, and amygdala, which process specific reward information but do not emit a global reward prediction error signal. A cooperation between the different reward signals may assure the use of specific rewards for selectively reinforcing behaviors. Among the other projection systems, noradrenaline neurons predominantly serve attentional mechanisms and nucleus basalis neurons code rewards heterogeneously. Cerebellar climbing fibers signal errors in motor performance or errors in the prediction of aversive events to cerebellar Purkinje cells. Most deficits following dopamine-depleting lesions are not easily explained by a defective reward signal but may reflect the absence of a general enabling function of tonic levels of extracellular dopamine. Thus dopamine systems may have two functions, the phasic transmission of reward information and the tonic enabling of postsynaptic neurons.

If the Brain does RL where is the Reward represented?



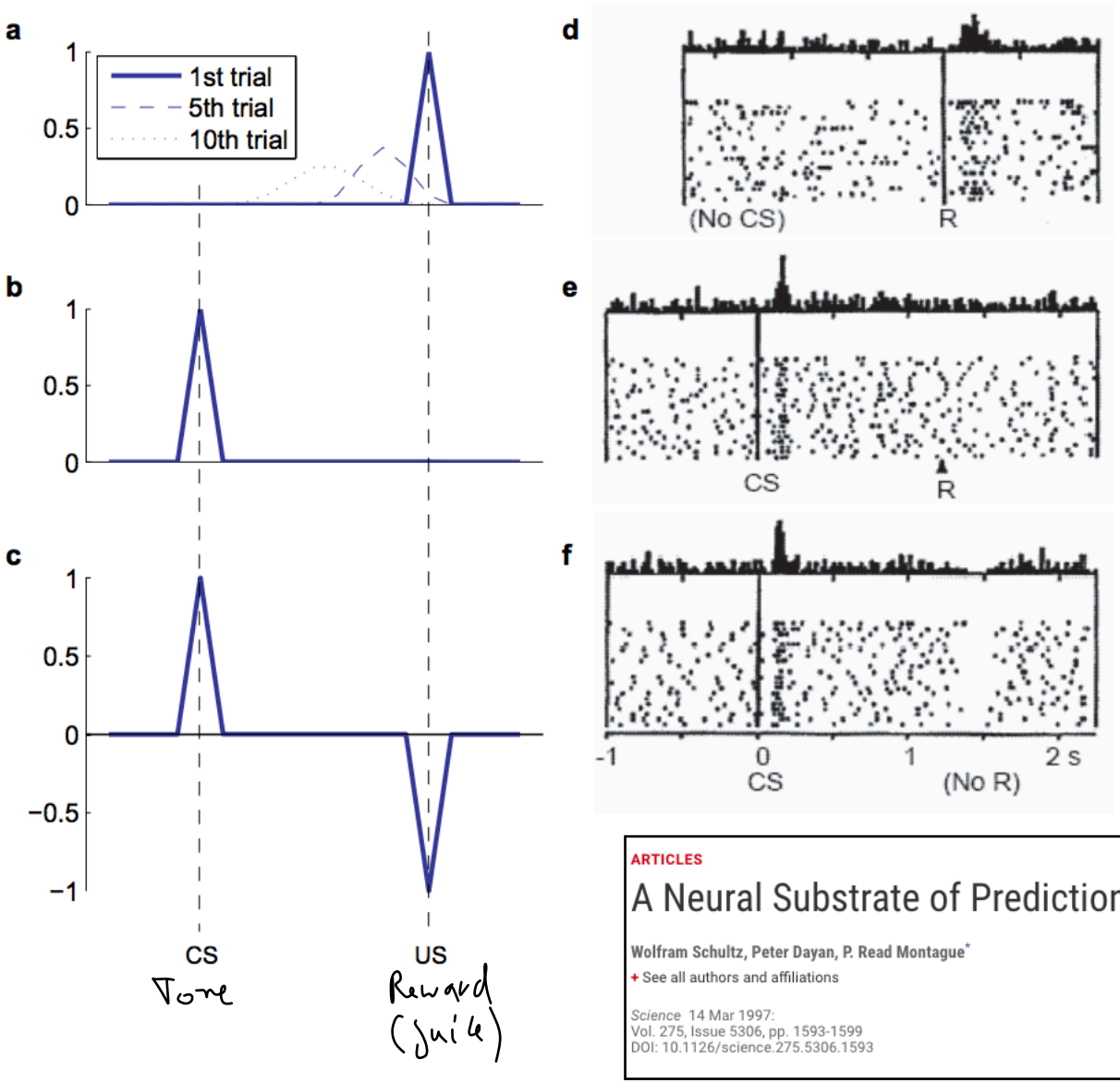
If the Brain does RL where is the Reward represented?



If the neocortex mostly performs unsupervised learning why does the VTA strongly project to almost all cortical areas and what is the effect of DA on a cortical neuron?

Discuss for 3 min !

Dopamine Encodes a Reward Prediction Error.



ARTICLES

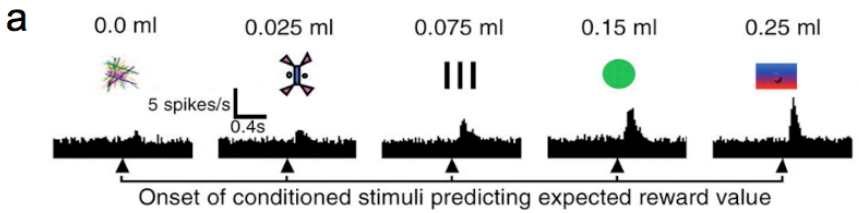
A Neural Substrate of Prediction and Reward

Wolfram Schultz, Peter Dayan, P. Read Montague*

+ See all authors and affiliations

Science 14 Mar 1997:
Vol. 275, Issue 5306, pp. 1593-1599
DOI: 10.1126/science.275.5306.1593

Dopamine Encodes a Reward Prediction Error.



REPORT

Adaptive Coding of Reward Value by Dopamine Neurons

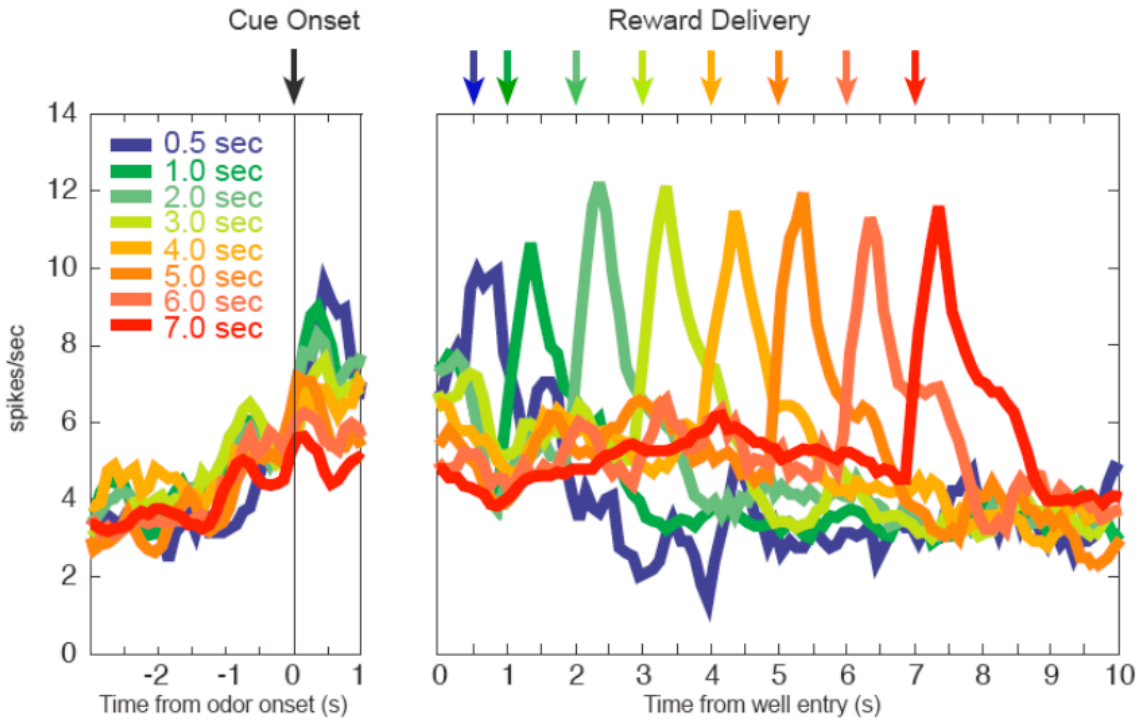
Philippe N. Tobler, Christopher D. Fiorillo*, Wolfram Schultz‡

* Present address: Department of Biology, Stanford University, Stanford, CA 94305–5125, USA.

+ See all authors and affiliations

Science 11 Mar 2005:
Vol. 307, Issue 5715, pp. 1642-1645
DOI: 10.1126/science.1105370

Dopamine Encodes a Reward Prediction Error.



frontiers in NEUROSCIENCE

REVIEW ARTICLE
published: 07 December 2011
doi: 10.3389/fnins.2011.00130

Impact of size and delay on neural activity in the rat limbic corticostriatal system

Matthew R. Roesch* and Daniel W. Bryden

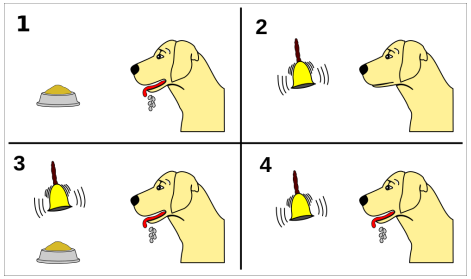
Department of Psychology and Program in Neuroscience and Cognitive Science, University of Maryland, College Park, MD, USA

The Rescorla Wagner Rule (1972)

The Idea:
Error-driven Learning derived from Classical Conditioning Experiments

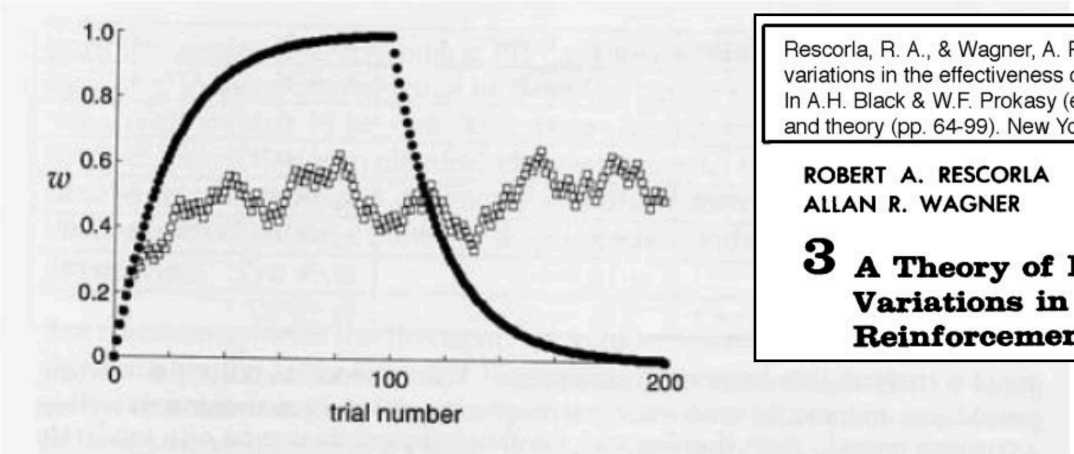
Change in value $V(s_t)$ is proportional to the difference between actual and predicted reward.

$$V(s_t) \leftarrow V(s_t) + \eta \, [\, R - \text{sum}(V(S_t)) \,]$$



Two assumptions/hypotheses:

- Learning is driven by the error signal (formalized as notion of surprise)
- Summation of predictors is linear.



Rescorla, R. A., & Wagner, A. R. (1972). A theory of Pavlovian conditioning: variations in the effectiveness of reinforcement and nonreinforcement. In A.H. Black & W.F. Prokasy (eds.), Classical conditioning II: current research and theory (pp. 64-99). New York: Appleton-Century-Crofts.

ROBERT A. RESCORLA
ALLAN R. WAGNER

3 A Theory of Pavlovian Conditioning: Variations in the Effectiveness of Reinforcement and Nonreinforcement

Temporal Difference (TD) Rule

Key Idea: Update the value (V) of the current state (s) based on the immediate reward and the estimated value of the next state.

New_Estimate = Old_Estimate + Step_Size [Target - Old_Estimate]

$$V(s_t) \leftarrow V(s_t) + \eta [R_t - V(S_t)]$$ RW Rule

$$V(s_t) \leftarrow V(s_t) + \eta [r_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$ TD(0) Rule

Previous estimate

Reward t+1

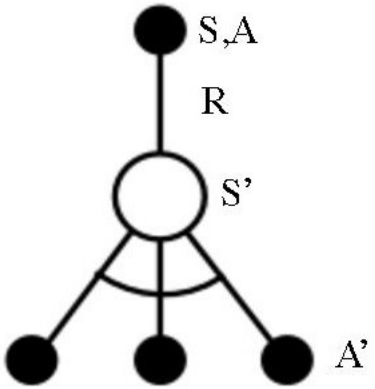
Discounted value on the next step

TD Target

From the TD Rule to Q-learning

$$V(s_t) \leftarrow V(s_t) + \eta \left[r_{t+1} + \gamma V(S_{t+1}) - V(S_t) \right]$$

Value of state S_t



Value of executing action A in state S

$$Q(S,A) \leftarrow Q(S,A) + \alpha \left(R + \gamma \max_{a'} Q(S',a') - Q(S,A) \right)$$

Previous estimate

Reward t+1

Discounted value on the next step

TD Target

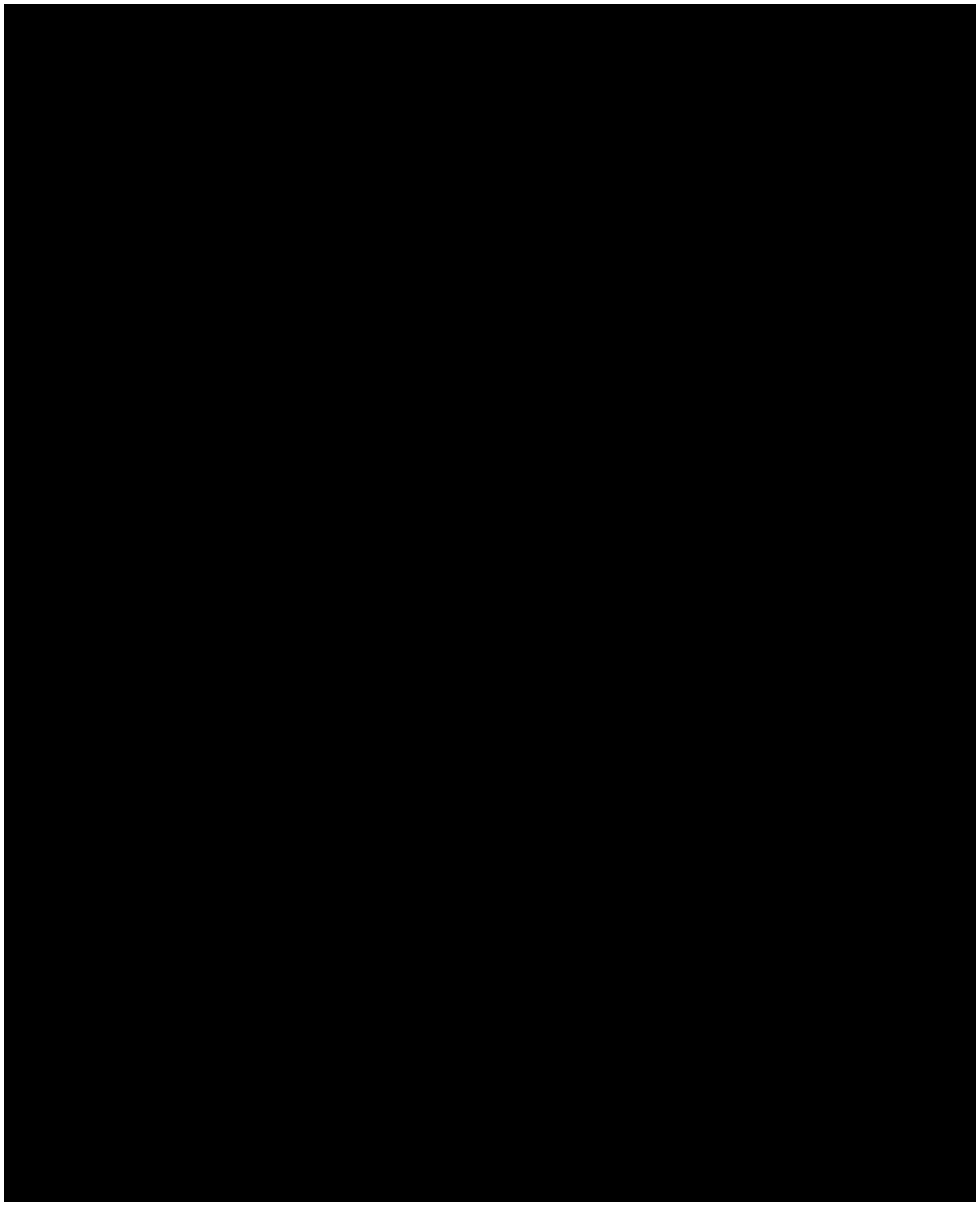
RL and Machine Learning.

Examples:

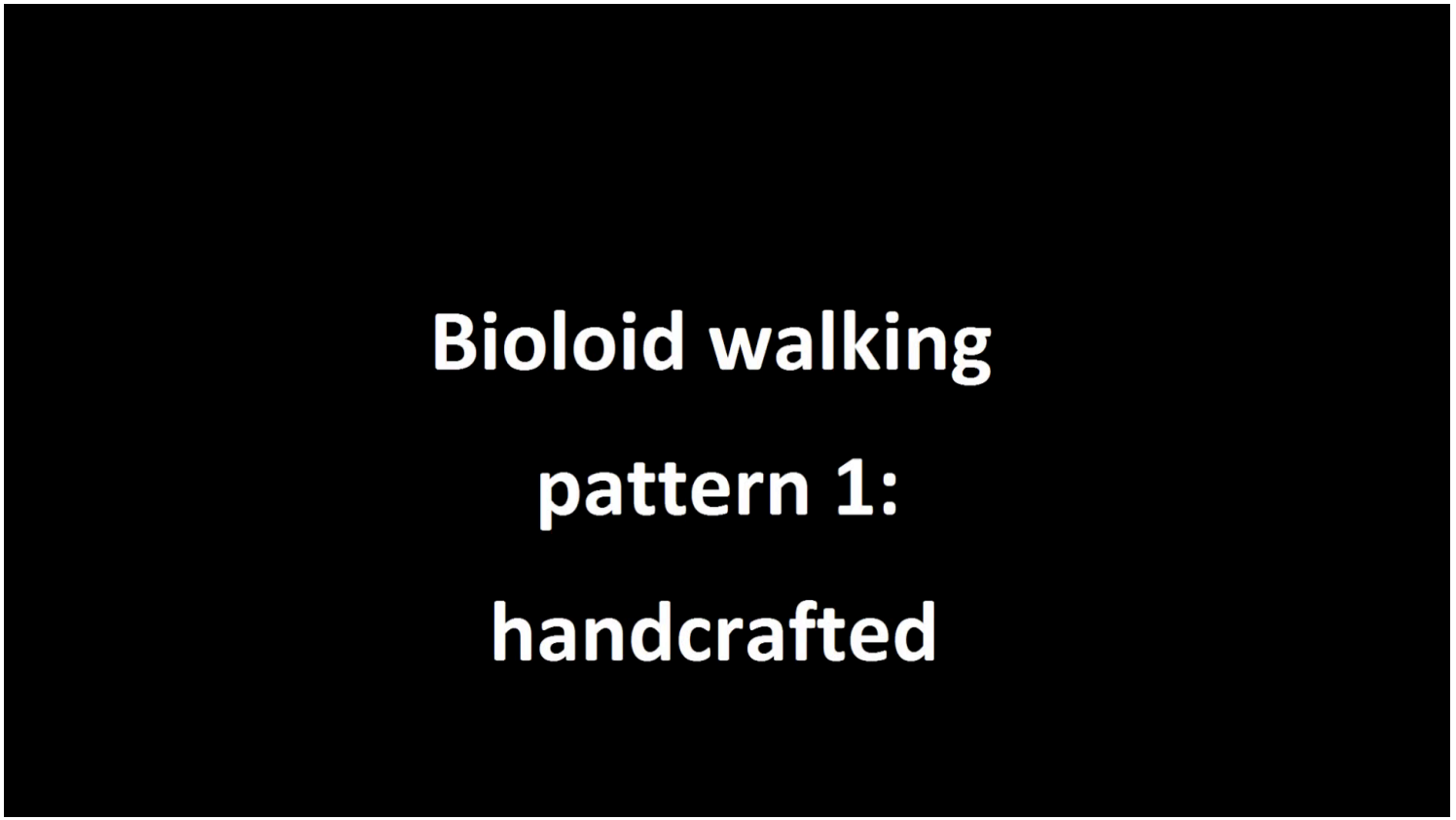
- **Play Atari games.**
- **Make a humanoid robot walk.**

- Defeat the world champion at Backgammon/GO.
- Manage an investment portfolio

RL Example #1: Playing Pong.

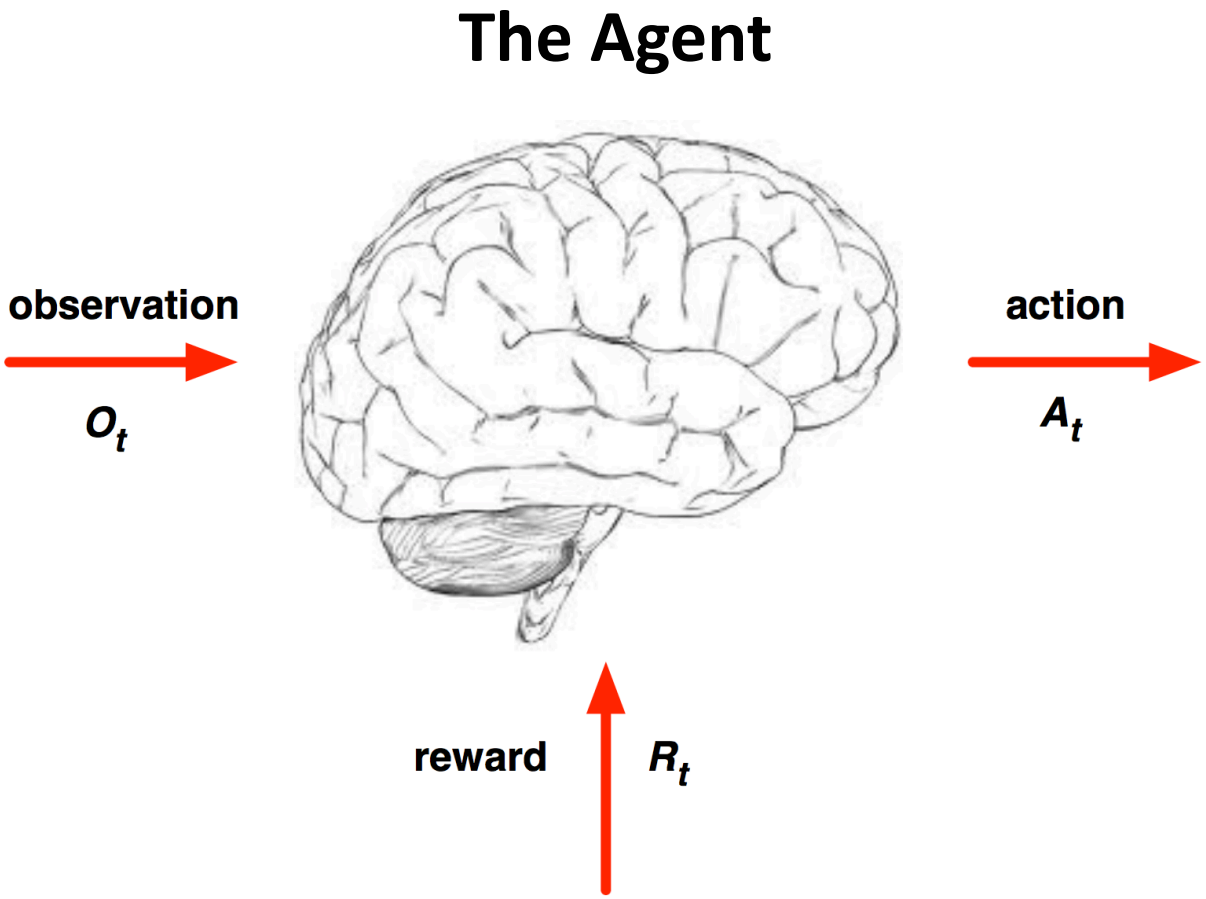


RL Example #3: Humanoid Robot Walking.



www.youtube.com/watch?v=O2rx4Bdwn24

The Basic Components of RL.



The Basic Components of RL.

Rewards:

- A reward R_t is a scalar feedback signal.
- Indicates how well the agent is doing at step t.
- The agent's job is to maximize cumulative reward.

Reinforcement learning is based on the reward hypothesis:

All goals can be described by the maximization of expected cumulative reward.

The Basic Components of RL.

Reward Examples:

- **Fly stunt maneuvers in an RC helicopter**
  + reward for following desired trajectory
  – reward for crashing
- **Defeat the world champion at Backgammon**
  +/– reward for winning/losing a game
- **Manage an investment portfolio**
  +/- reward for each $ in the bank
- **Make a humanoid robot walk**
  + reward for forward motion
  – reward for falling over
- **Play Atari games better than humans**
  +/– reward for increasing/decreasing score

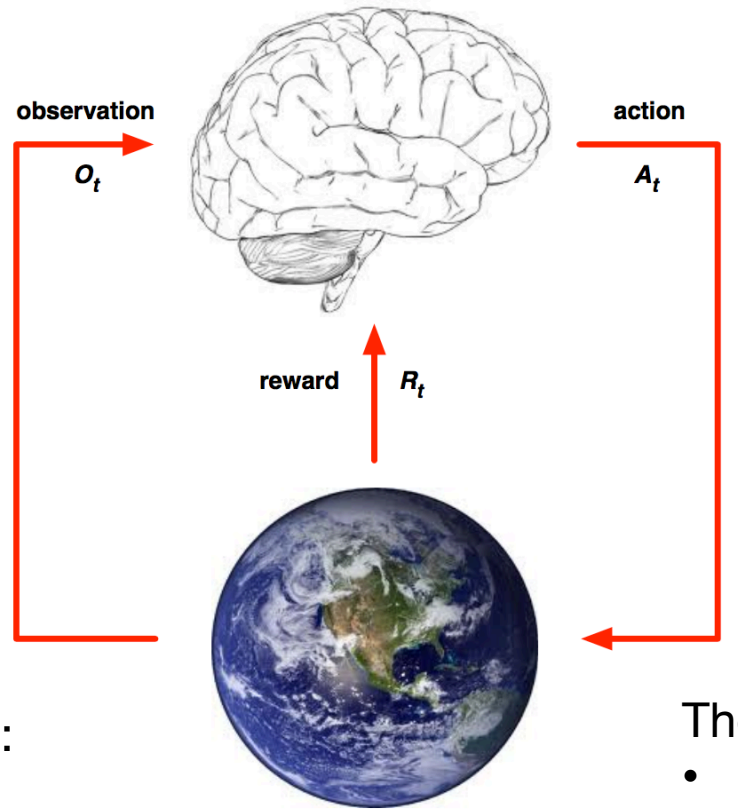
Reward and Decision Making.

- **Goal:** Select actions to maximize total future rewards.
- Actions may have long term consequences.
- Reward may be delayed.
- It may be better to sacrifice immediate reward to gain more long-term reward.

Example:

- A financial investment (may take months to mature).
- Re-fuelling a helicopter (might prevent a crash in several hours).
- Blocking opponent an move (might help winning chances many moves from now).

The Basic Components of RL.



At each step t the agent:

- Executes action A_t
- Receives observation O_t
- Receives scalar reward R_t

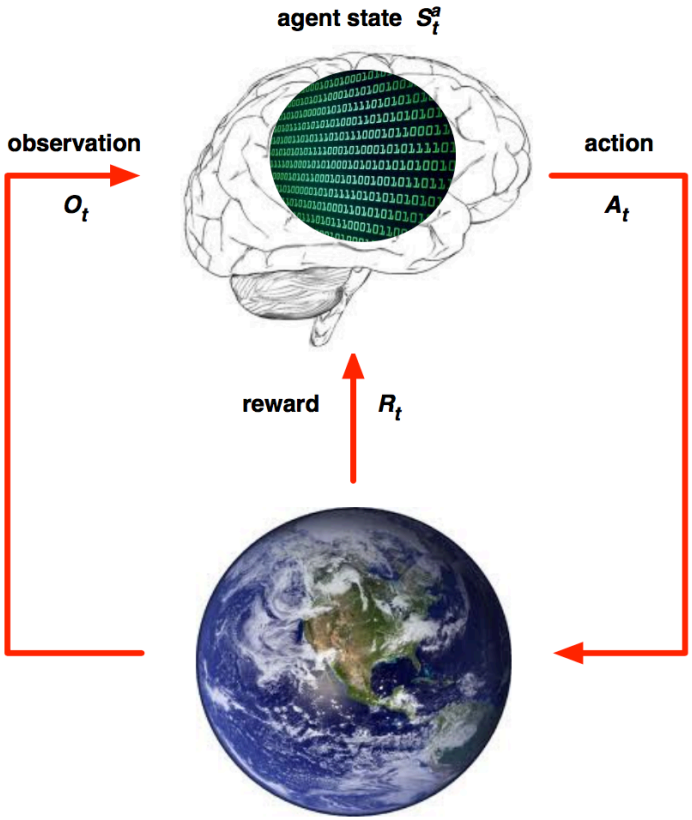
The environment:

- Receives action A_t
- Emits observation O_{t+1}
- Emits scalar reward R_{t+1}
-

History $H_t = A_1\ O_1\ R_1, \ldots, A_t\ O_t\ R_t$

$S(t) = f(H_t)$

The Agent State.



- The agent state S_t^a is the agent's internal representation,
  - whatever information the agent uses to pick the next action
  - it is the information used by reinforcement learning algorithms
- It can be any function of the history: $S_t^a = f(H_t)$.

The Information State.

An information state (a.k.a. Markov state) contains all useful information from the history.

Definition: In a 1st order Markov chain a state S_t is Markov if and only if

$$P[S_{t+1} \mid S_t] = P[S_{t+1} \mid S_1, \ldots, S_t]$$

- "The future is independent of the past given the present"
  $H_{1:t} \rightarrow S_t \rightarrow H_{t+1:\infty}$
- Once the state is known, the history may be thrown away.
- The state is a sufficient statistic of the future.
- The environment state S_t^e is Markov.
- The history H_t is Markov.

The Lab Rat Example.



The agent state defines what is next….

- What if agent state = last 3 items in sequence?
- What if agent state = counts for lights, bells and levers?
- What if agent state = complete sequence?

The Basic Components of an RL Agent

An RL agent may include one or more of these components:

- **Policy:** The agent's behavior function.
- **Value function:** How good is each state and/or action?
- **Model:** The agent's representation of the environment.

What is a Policy?

- A policy defines the agent's behavior.
- It is a map from state to action.
- Deterministic policy: $a = \pi(s)$
- Stochastic policy: $\pi(a|s) = P[A_t = a|S_t = s]$

What is a Value Function?

- Value function is a prediction of future rewards.
- It is used to evaluate the goodness/badness of states…
- and to select between actions.

$$v_\pi(s) = E_\pi \left(R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots | S_t = s \right)$$

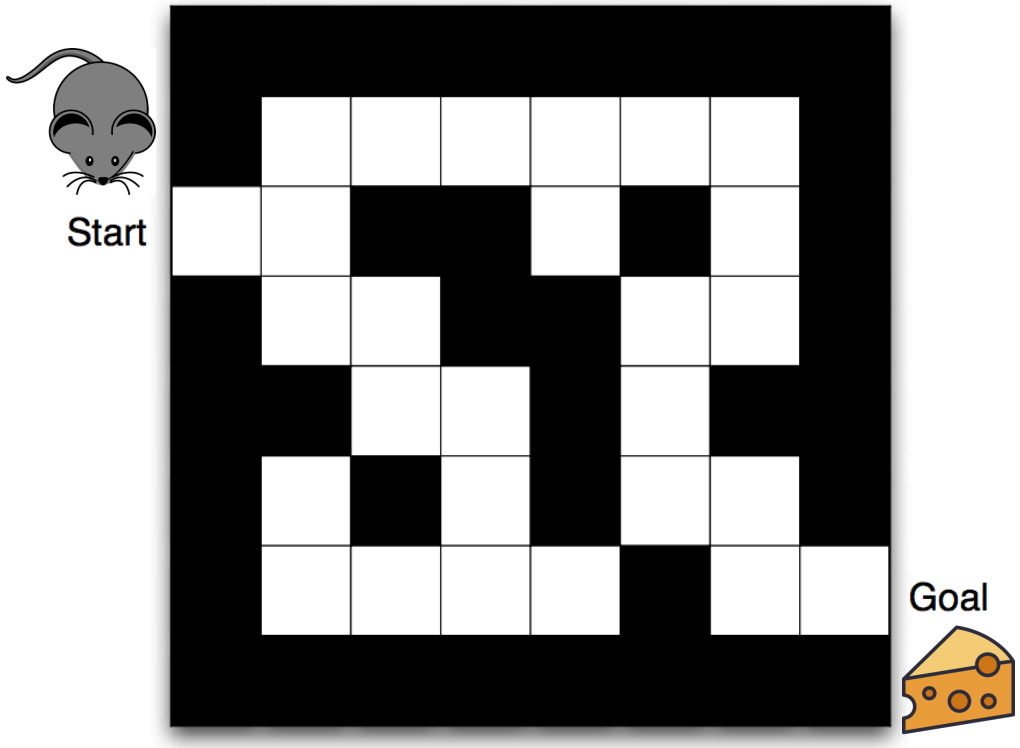
The Model.

- A model predicts what the environment will do next.
- P predicts the next state.
- R predicts the next (immediate) reward.

$$P_{ss'}^{a} = P\,[S_{t+1} = s' \,|\, S_t = s,\, A_t = a]$$

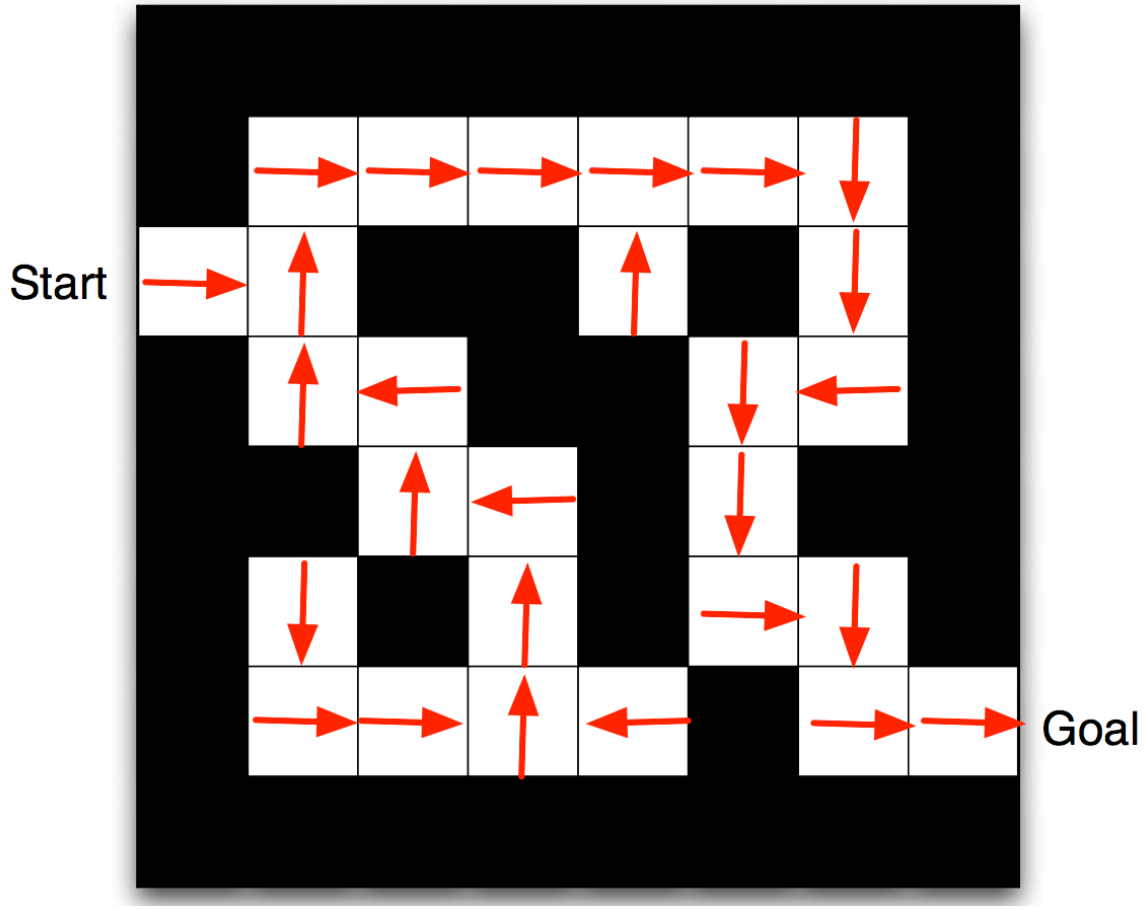
$$R_s^a = E\,[R_{t+1} \,|\, S_t = s,\, A_t = a]$$

Maze RL Example



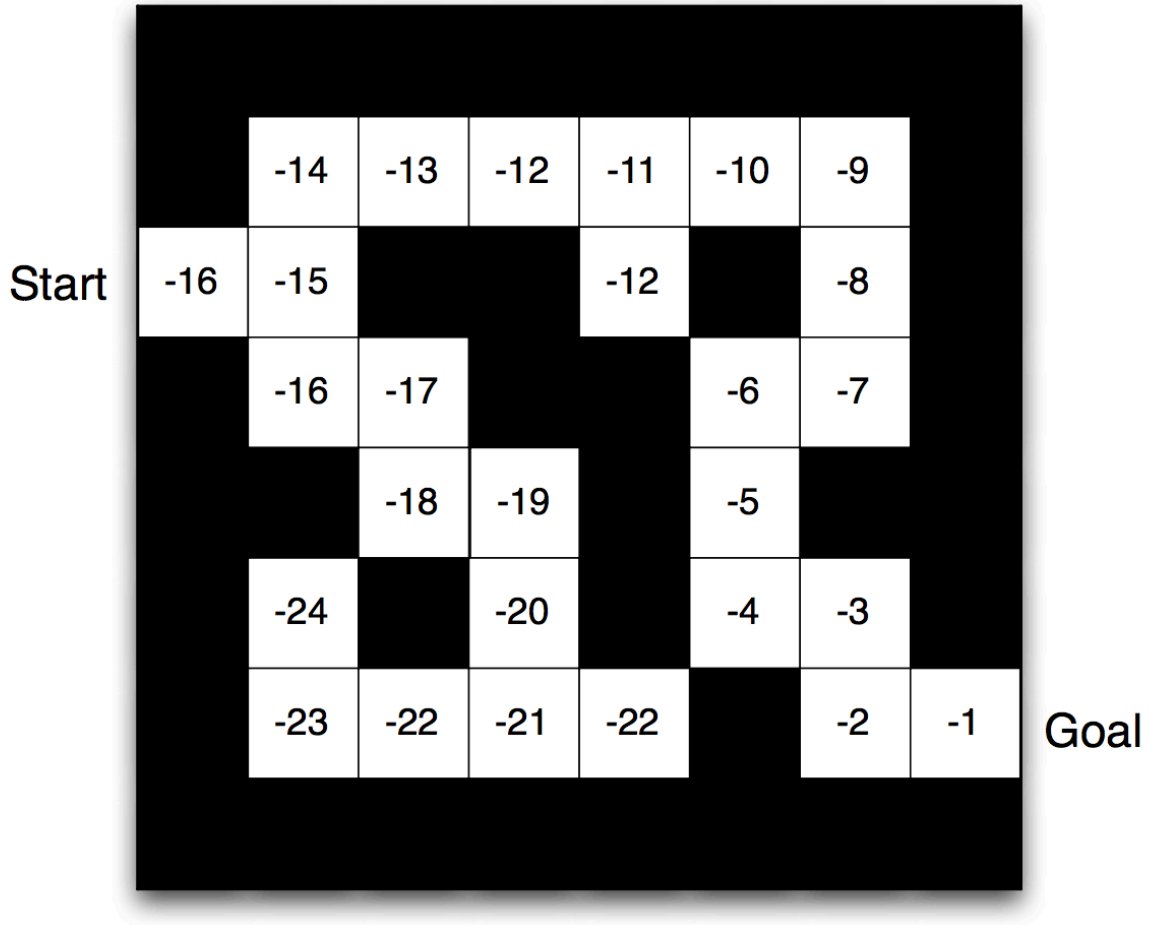
- Rewards: -1 per time-step
- Actions: N, E, S, W
- States: Agent's locations

Maze RL Example: Policy



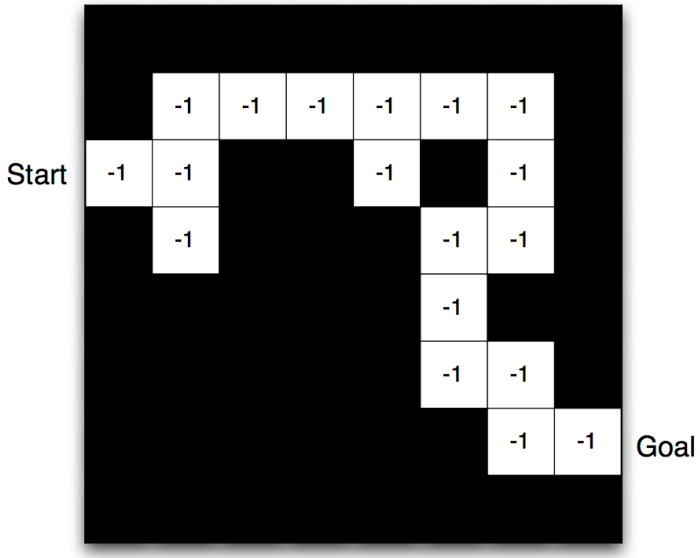
- Arrows represent policy π(s) for each state s

Maze RL Example: State Value



- Numbers represent value $v_\pi(s)$ of each state s

Maze RL Example: Model of the Environment



- Agent may have an internal model of the environment.
- Dynamics: How actions change the state.
- Rewards: How much reward from each state.
- The model may be imperfect .
- Grid layout represents transition model $P_{ss'}^{a}$.
- Numbers represent immediate reward R_s^a from each state s.

RL Sub-types.

Value Based

- No Policy (Implicit)
- **Value Function**

Policy Based

- **Policy**
- No Value Function

Model Free

- **Policy and/or Value Function**
- No Model

Model Based

- **Policy and/or Value Function**
- **Model**

RL and Planning

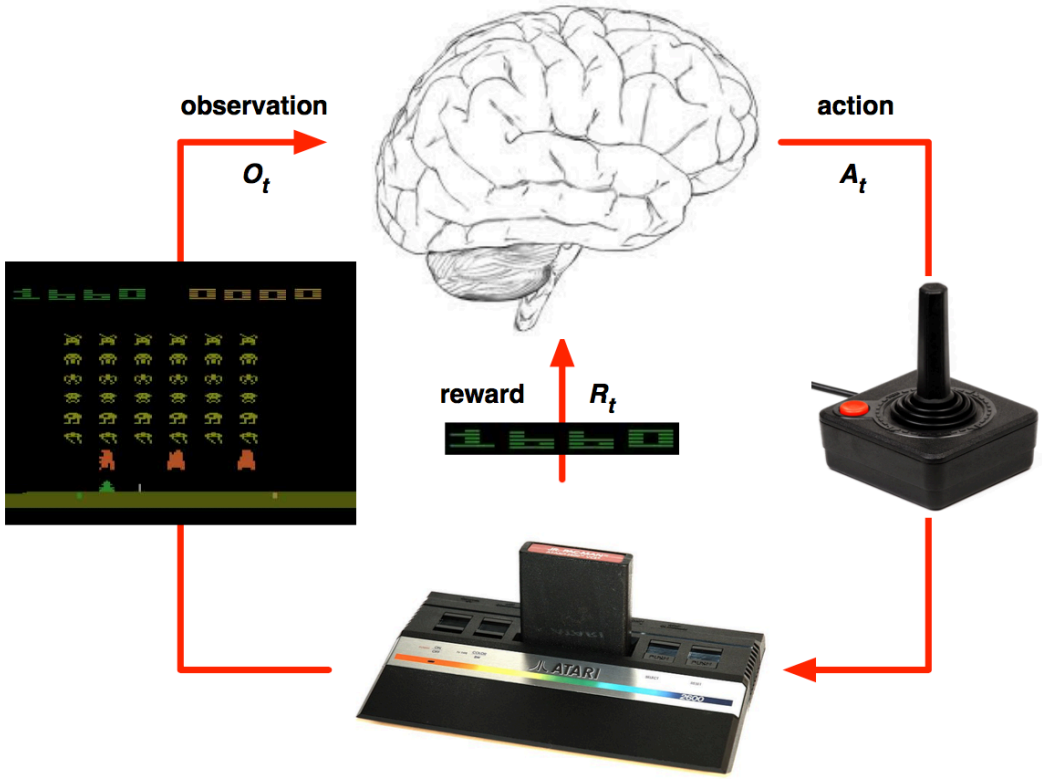
Reinforcement Learning:

- The environment is initially unknown.
- The agent interacts with the environment.
- The agent improves its policy or value function.

Planning:

- A model of the environment is known.
- The agent performs computations with its model (without any external interaction).
- The agent improves its policy.

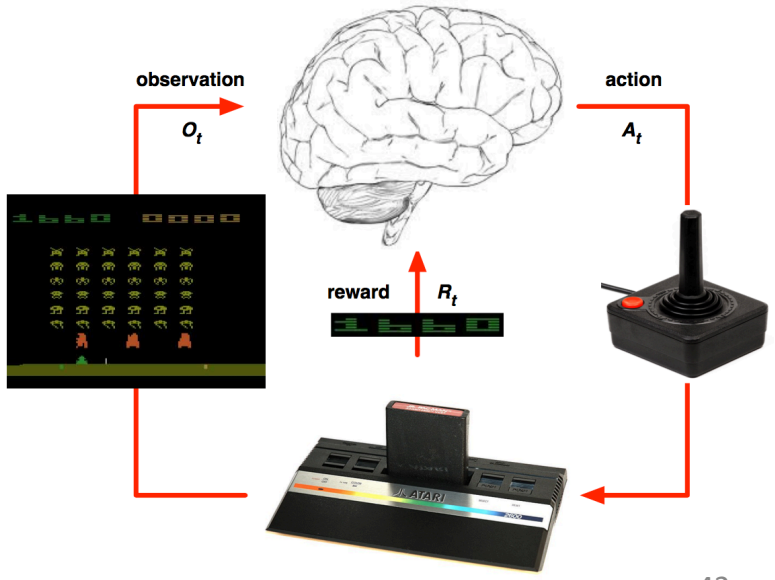
RL Example: Playing Atari Games



- Rules of the game are unknown.
- Learn directly from interactive game-play.
- Pick actions on joystick, see pixels and scores.

RL Example: Playing Atari Games - Planning

- Rules of the game are known.
- Can query emulator as perfect model inside agent's brain.
- If I take action a from state s:
  - What would the next state be?
  - What would the score be?
- Plan ahead to find optimal policy
  e.g. tree search.



Exploration and Exploitation.

- Reinforcement learning is ‘trial-and-error’ learning.
- The agent should discover a good policy…
- from its experiences of the environment…
- without losing too much reward along the way.

- **Exploration** finds more information about the environment.
- **Exploitation** exploits known information to maximize reward.
  It is usually important to explore as well as exploit.

Examples of Exploration and Exploitation.

- **Restaurant**
  - Exploitation: Always go to your favorite restaurant.
  - Exploration: Try a new restaurant.
- **Online Advertisements**
  - Exploitation: Show the most successful advert.
  - Exploration: Show a different advert.
- **Oil Drilling**
  - Exploitation: Drill at the best known location.
  - Exploration: Drill at a new location.

MCs, MRPs and MDPs.

- Markov decision processes formally describe an environment for reinforcement learning.
- Where the environment is fully observable.
  - i.e. The current state completely characterizes the process.
- Almost all RL problems can be formalized as MDPs:
  - Partially observable problems can be converted into MDPs.
  - One Armed Bandits are MDPs with one state.

MCs, MRPs and MDPs.

Definition: A state S_t is Markov if and only if

$$P[S_{t+1} \mid S_t] = P[S_{t+1} \mid S_1, \ldots, S_t]$$

- The state captures all relevant information from the history.
- Once the state is known, the history may be thrown away.
- The state is a sufficient statistic of the future.

MCs, MRPs and MDPs.

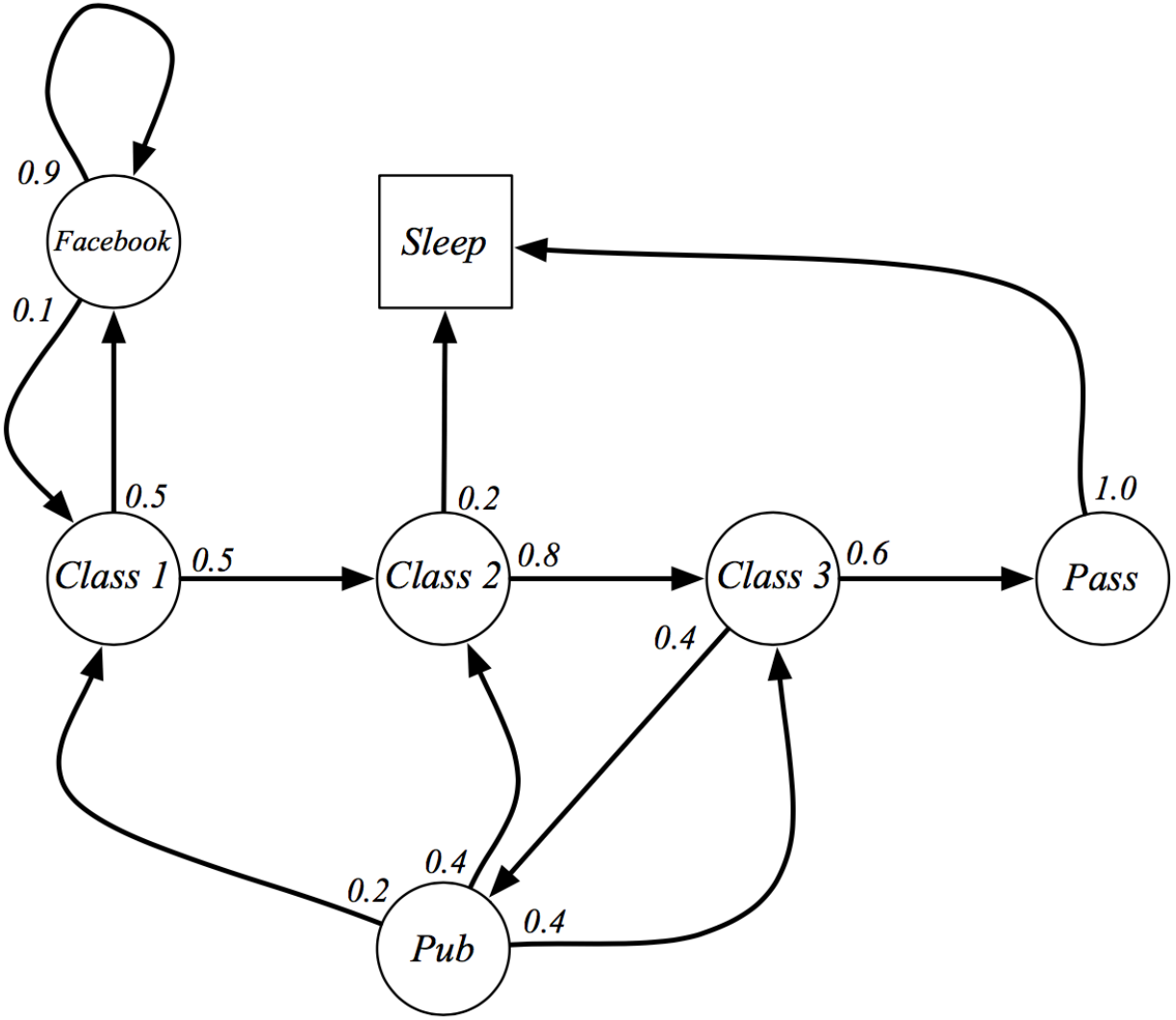
For a 1st order Markov chain with state s and successor state s′ and the state transition probability is defined by:

$$P_{ss'} = P\,[\,S_{t+1} = s' \mid S_t = s\,]$$

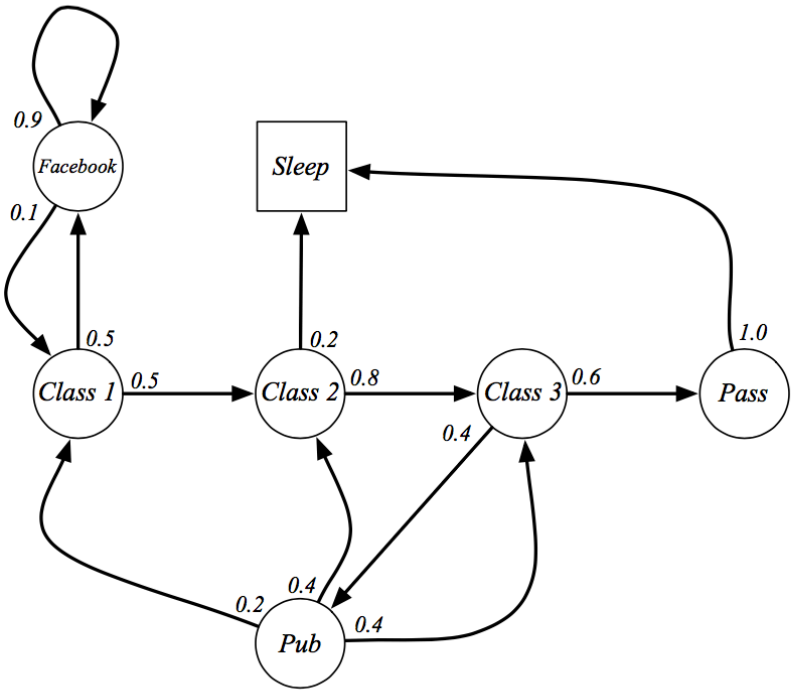
State transition matrix P defines transition probabilities from all states s to all successor states s′.

$$\mathcal{P} = \begin{array}{c} \\ \text{from} \end{array} \begin{array}{c} \text{to} \\ \begin{bmatrix} \mathcal{P}_{11} & \cdots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{n1} & \cdots & \mathcal{P}_{nn} \end{bmatrix} \end{array}$$

The Student Markov Chain (MC)



The Student Markov Chain (MC)

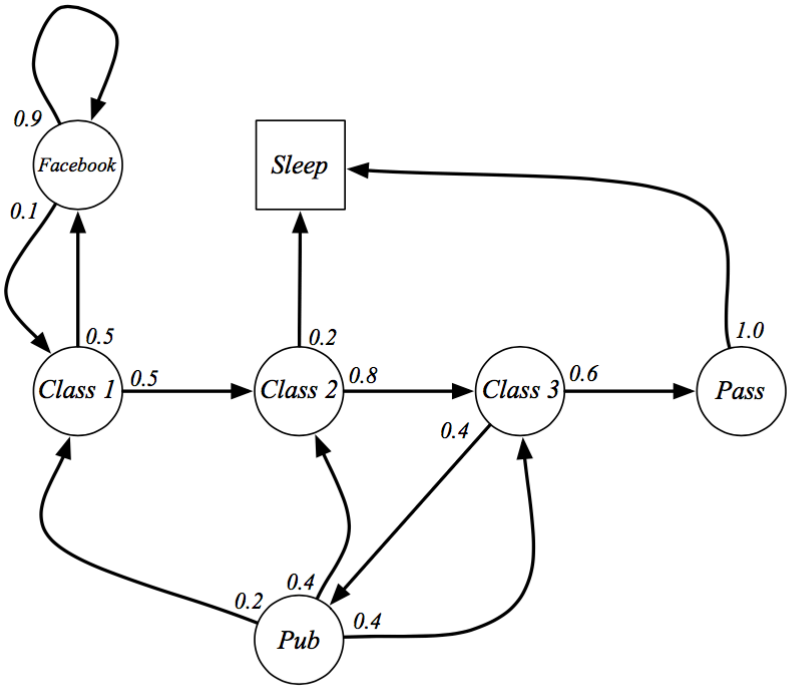


Sample episodes for Student Markov Chain starting from $S_1 = \text{C1}$

$$S_1, S_2, ..., S_T$$

- C1 C2 C3 Pass Sleep
- C1 FB FB C1 C2 Sleep
- C1 C2 C3 Pub C2 C3 Pass Sleep
- C1 FB FB C1 C2 C3 Pub C1 FB FB FB C1 C2 C3 Pub C2 Sleep

The Student Markov Chain (MC).



State transition matrix

$$\mathcal{P} = \begin{array}{c} \\ C1 \\ C2 \\ C3 \\ Pass \\ Pub \\ FB \\ Sleep \end{array} \begin{array}{c} \begin{array}{ccccccc} C1 & C2 & C3 & Pass & Pub & FB & Sleep \end{array} \\ \begin{bmatrix} & 0.5 & & & & 0.5 & \\ & & 0.8 & & & & 0.2 \\ & & & 0.6 & 0.4 & & \\ & & & & & & 1.0 \\ 0.2 & 0.4 & 0.4 & & & & \\ 0.1 & & & & & 0.9 & \\ & & & & & & 1 \end{bmatrix} \end{array}$$

The Markov Reward Process (MRP)

A Markov reward process is a Markov chain with reward values.

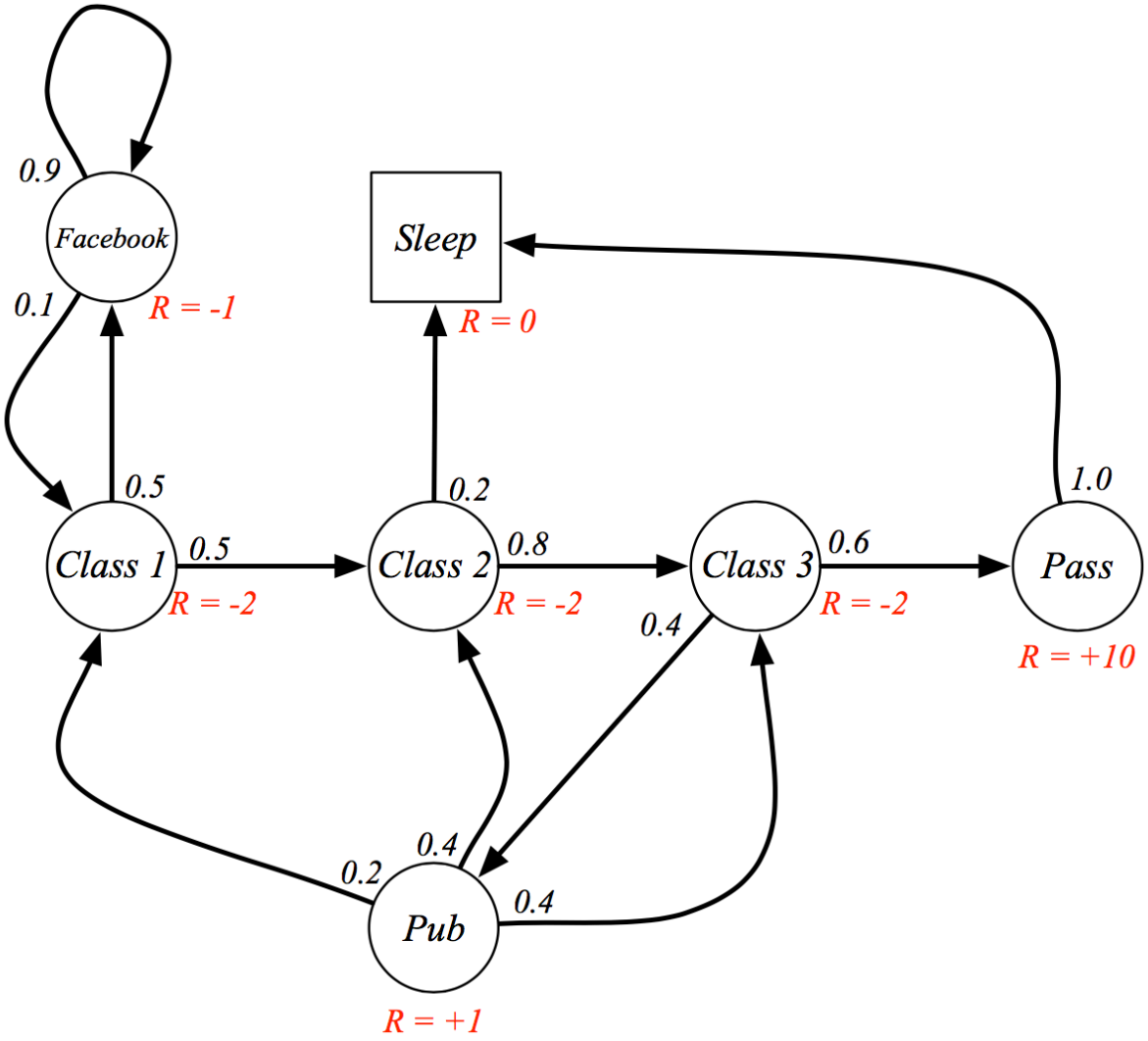
Definition: A Markov Reward Process is a tuple ⟨**S, P, R, γ**⟩

- S is a finite set of states
- P is a state transition probability matrix, $P_{ss'} = P[S_{t+1} = s' | S_t = s]$
- R is a reward function, $R_s = E[R_{t+1} | S_t = s]$
- γ is a discount factor, $\gamma \in [0, 1]$

Why discount future rewards?

- Mathematically convenient to discount rewards.
- Avoids infinite returns in cyclic Markov processes.
- Uncertainty about the future may not be fully represented.
- If the reward is financial, immediate rewards may earn more interest than delayed rewards.
- Animal/human behavior shows preference for immediate rewards.
- It is sometimes possible to use un-discounted Markov reward processes (i.e. $\gamma = 1$), e.g. if all sequences terminate.

Example of a Student MRP



Reward and Return

Return Definition:

The *return* G_t is the total discounted reward from time-step t.

$$G_t = R_{t+1} + \gamma R_{t+2} + \ldots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- The discount $\gamma \in [0, 1]$ is the present value of future rewards
- The value of receiving reward R after k + 1 time-steps is γ^k R .
- This values immediate reward above delayed reward.
  - γ close to 0 leads to ”myopic” evaluation
  - γ close to 1 leads to ”far-sighted” evaluation

The MRP Value Function.

The value function v(s) gives the long-term value of state s.

Definition: The state value function v(s) of an MRP is the expected return starting from state s.

$$v(s)=E\left[G_t \mid S_t = s\right]$$

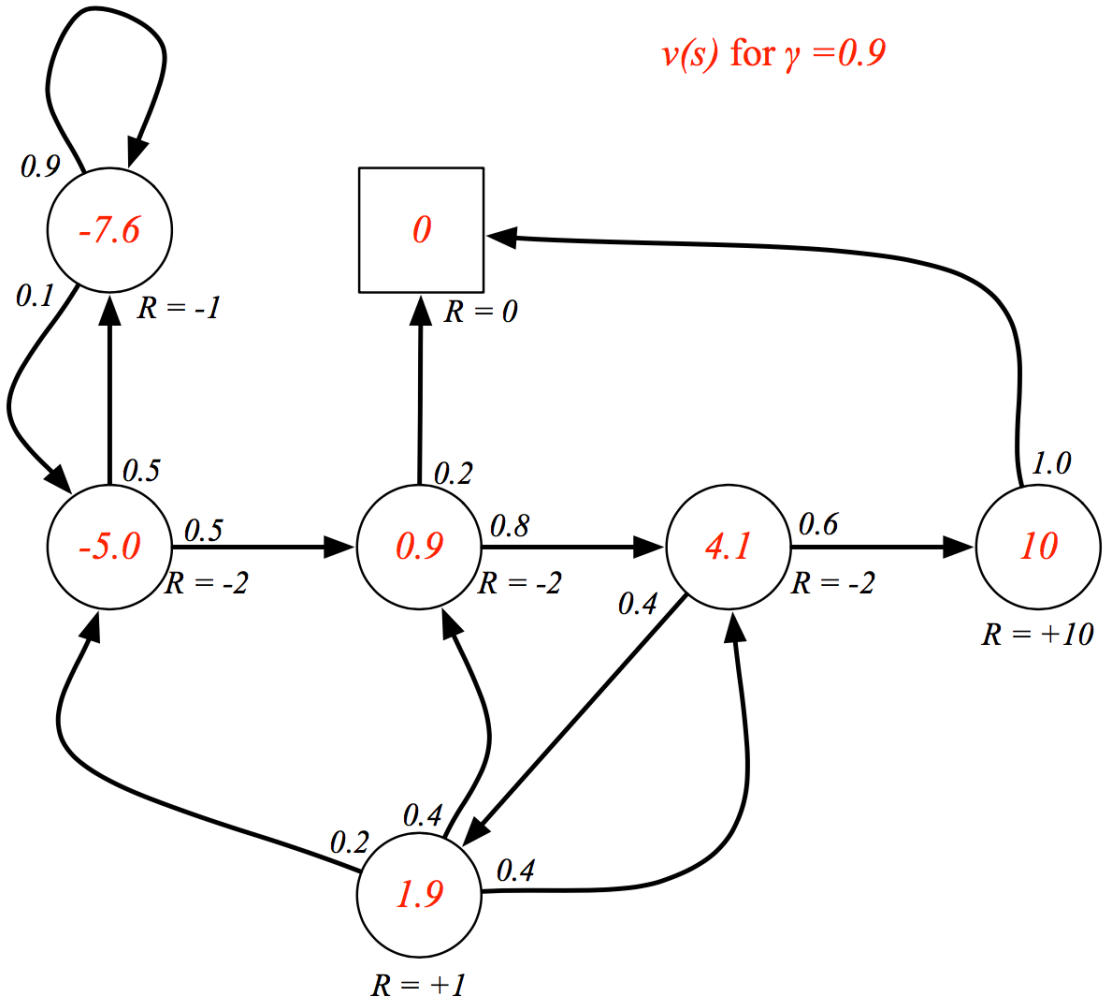
Example pf Student MRP Returns.

Sample returns for Student MRP:
Starting from $S_1 = \text{C1}$ with $\gamma = \frac{1}{2}$

$$G_1 = R_2 + \gamma R_3 + ... + \gamma^{T-2} R_T$$

C1 C2 C3 Pass Sleep	$v_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 10 * \frac{1}{8}$	$=$	-2.25
C1 FB FB C1 C2 Sleep	$v_1 = -2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16}$	$=$	-3.125
C1 C2 C3 Pub C2 C3 Pass Sleep	$v_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 1 * \frac{1}{8} - 2 * \frac{1}{16} \dots$	$=$	-3.41
C1 FB FB C1 C2 C3 Pub C1 ... FB FB FB C1 C2 C3 Pub C2 Sleep	$v_1 = -2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16} \dots$	$=$	-3.20

Example of a Student MRP



Bellmann Expectation Equation for MRPs

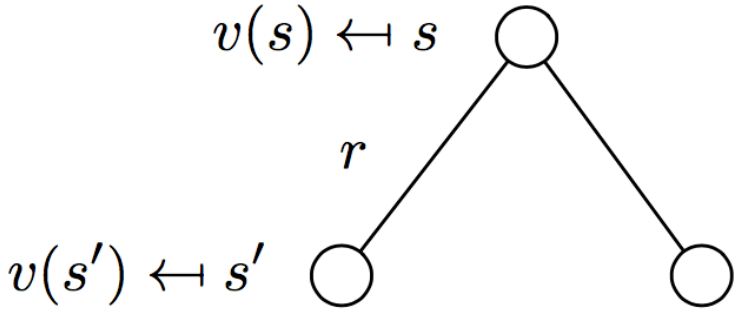
The value function can be decomposed into two parts:

- Immediate reward R_{t+1}.
- Discounted value of successor state $\gamma\, v(S_{t+1})$.

$$
\begin{aligned}
v(s) &= E\,[G_t \,|S_t = s] \\
&= E\,[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \,|S_t = s] \\
&= E\,[R_{t+1} + \gamma\,(R_{t+2} + \gamma R_{t+3} + \ldots) \,|S_t = s] \\
&= E\,[R_{t+1} + \gamma\, \mathbf{G_{t+1}} \,|S_t = s] \\
&= E\,[R_{t+1} + \gamma\, v(S_{t+1}) \,|S_t = s]
\end{aligned}
$$

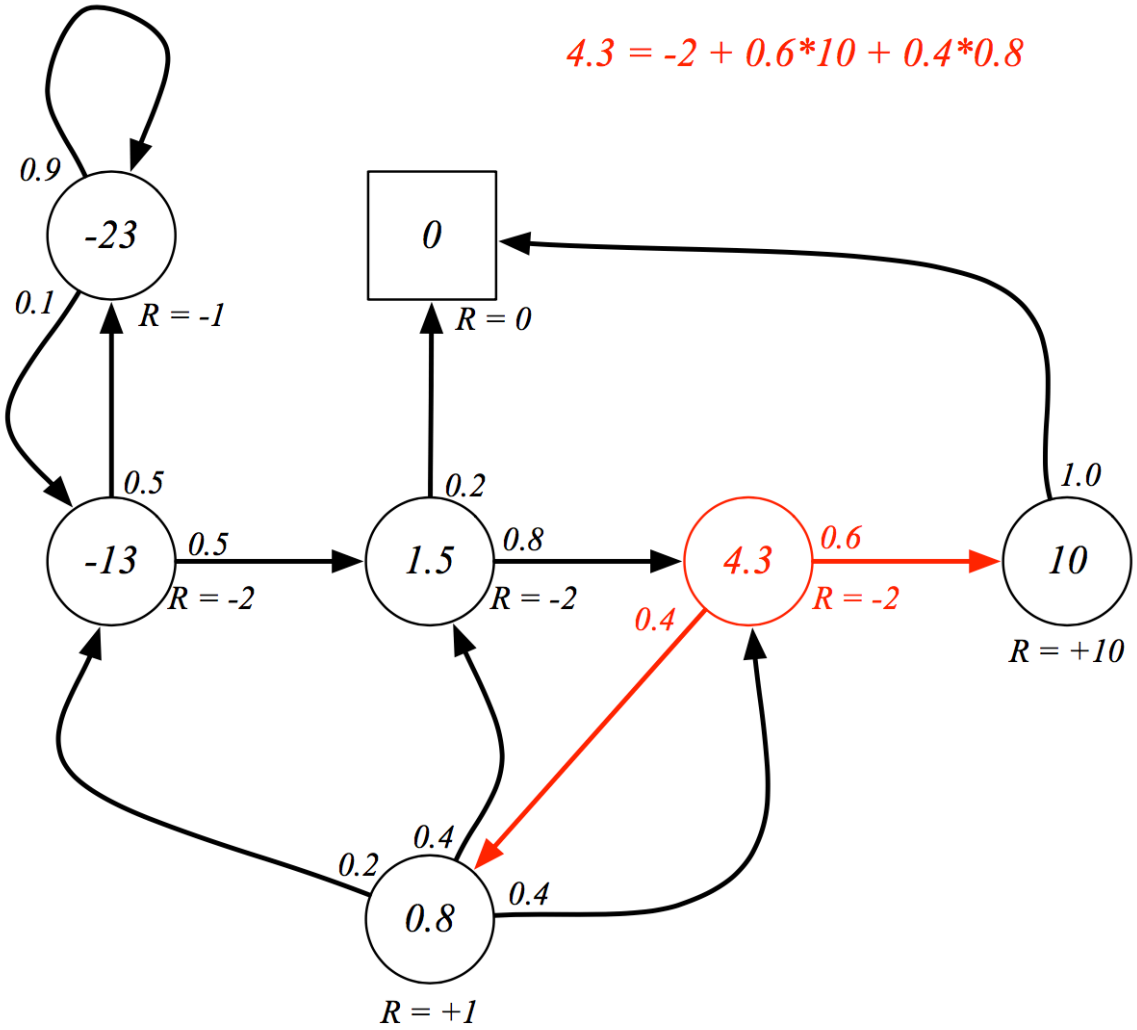
Bellmann Expectation Equation for MRPs

$$v(s) = \mathbb{E}\left[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s\right]$$



$$v(s) = \mathcal{R}_s + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'} v(s')$$

Example: Bellmann Expectation Equation for Student MRP



The Markov Decision Process (MDP)

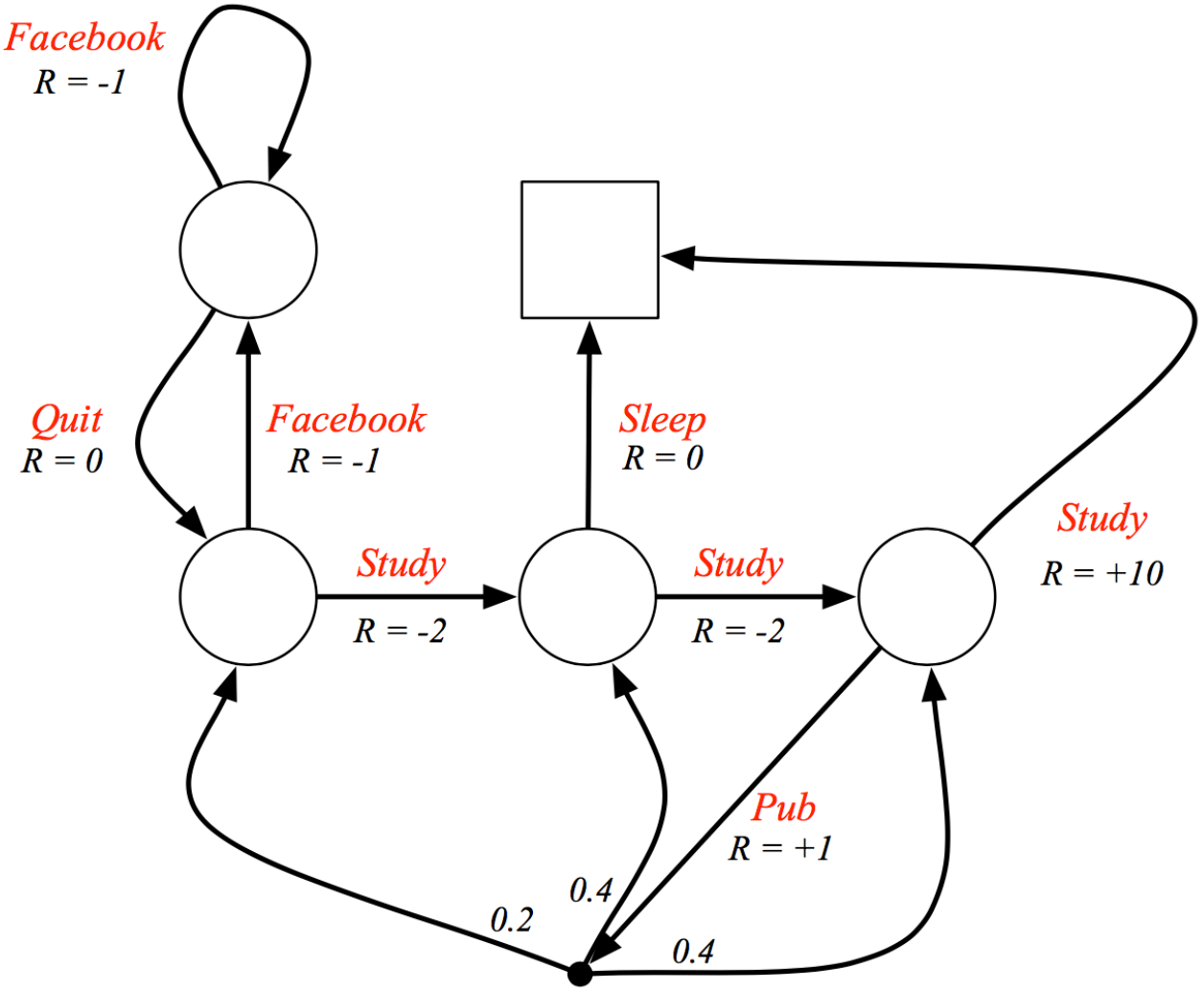
- Markov decision process (MDP) is a Markov reward process with decisions. It is still an environment in which all states are Markov.

Definition: A Markov Decision Process is a tuple $\langle \boldsymbol{S, A, P, R, \gamma} \rangle$

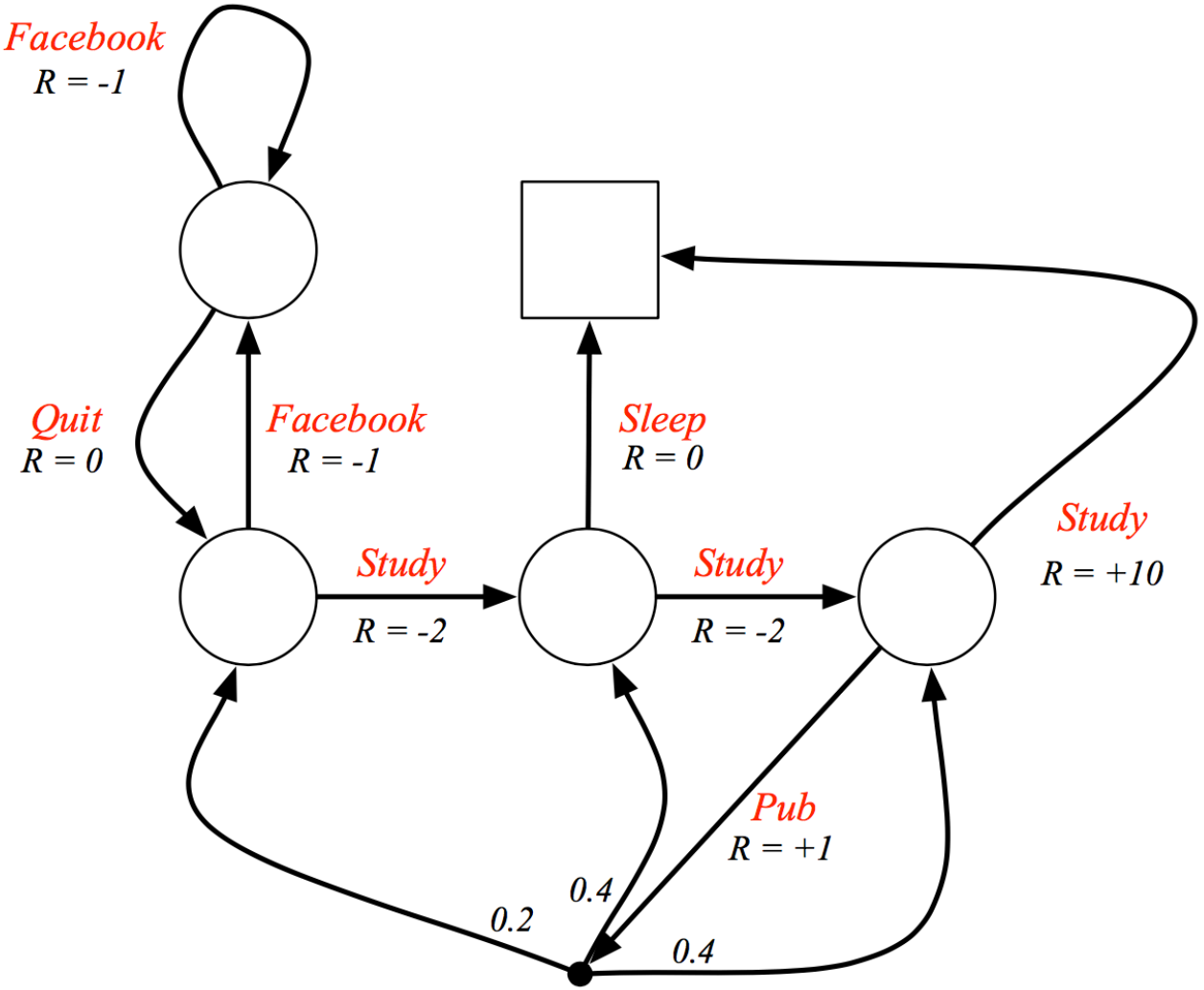
- S is a finite set of states.
- A is a finite set of actions.
- P is a state transition probability matrix,

  $P_a = P\,[S_{t+1} = s' | S_t = s,\, A_t = a]$
- R is a reward function, $R_s^a = E\,[R_{t+1} \mid S_t = s,\, A_t = a]$
- γ is a discount factor $\gamma \in [0, 1]$.

Example of a Student MDP



How do we find the actions that provide the max. return?



Express Value Function Based on Actions.

- **Definition 1:** The state-value function $v_\pi(s)$ of an MDP is the expected return starting from state s, and then following policy π.

$$v_\pi(s)=E_\pi[\, G_t \,|S_t = s]$$

- **Definition 2:** The action-value function $q_\pi(s,a)$ is the expected return starting from state s, taking action a, and then following policy π.

$$q_\pi(s,a)=E_\pi[\, G_t \,|S_t = s,\, A_t = a]$$

The Bellmann Expectation Equation

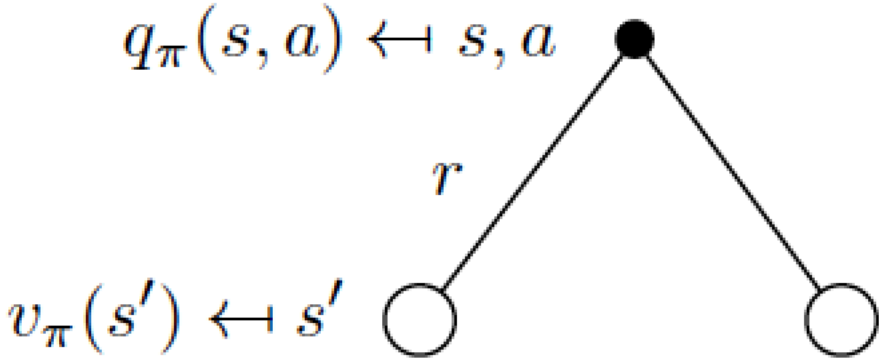
The state-value function can again be decomposed into immediate reward plus discounted value of successor state.

$$v_\pi(s) = E_\pi [R_{t+1} + \gamma\, v_\pi(S_{t+1}) \mid S_t = s]$$

The action-value function can similarly be decomposed.

$$q_\pi(s,a) = E_\pi [R_{t+1} + \gamma\, q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

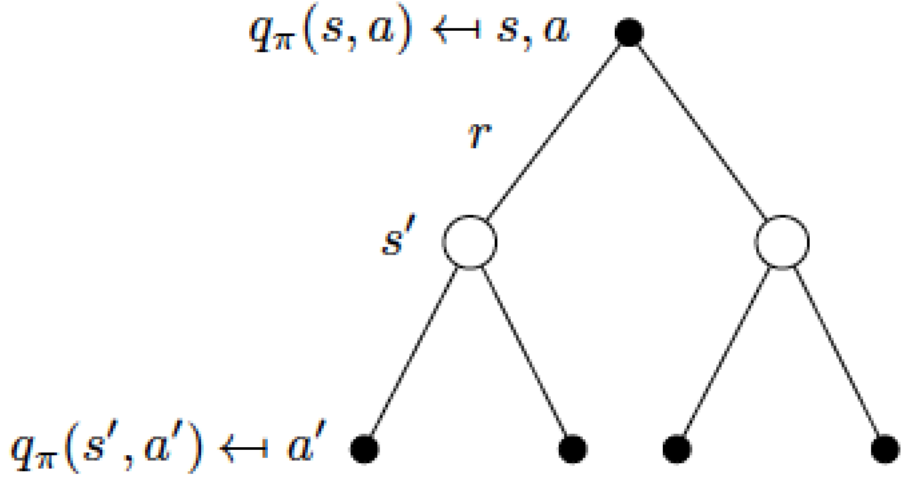
The Bellmann Expectation Equation or v_π and q_π



$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s')$$

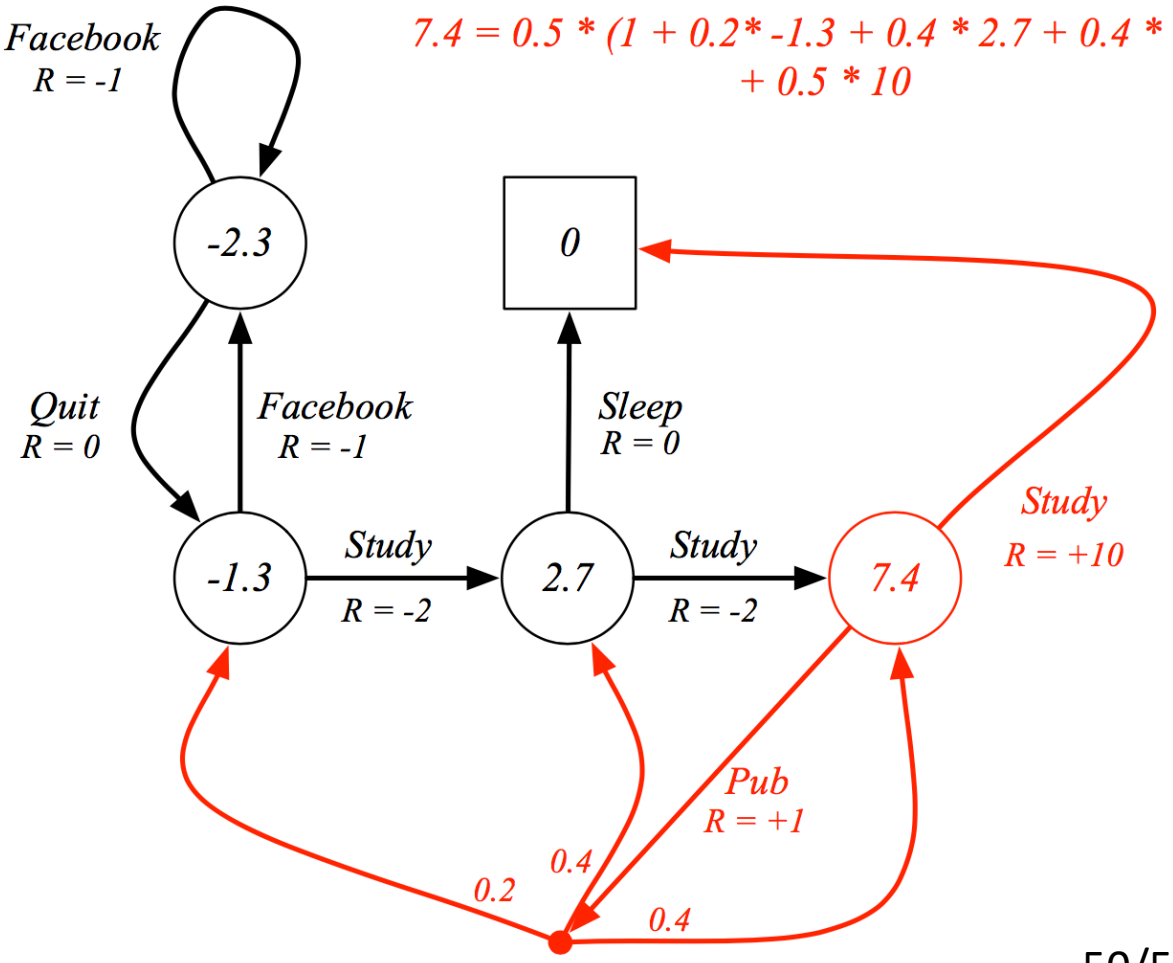
Sum over possible actions!

The Bellmann Expectation Equation or v_π and q_π



$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') q_\pi(s',a')$$

The Bellmann Expectation Equation for Student MDP



50/50 choice policy

How do we find the optimal action value function?

A 50/50 choice policy might not be optimal!

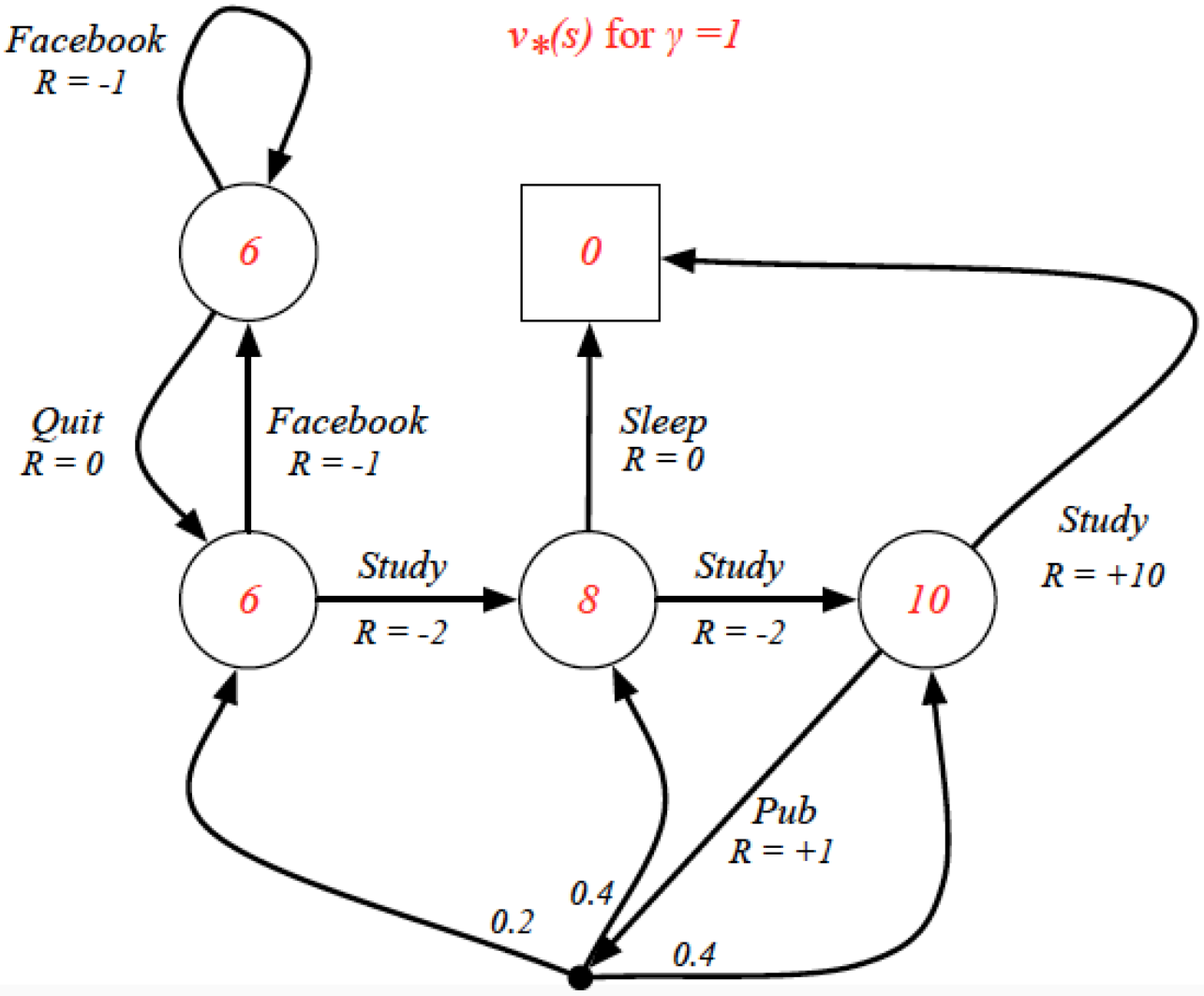
Definition 1: The optimal state-value function $v_*(s)$ is the maximum value function over all policies.

$$v_*(s) = \max v_\pi(s)$$

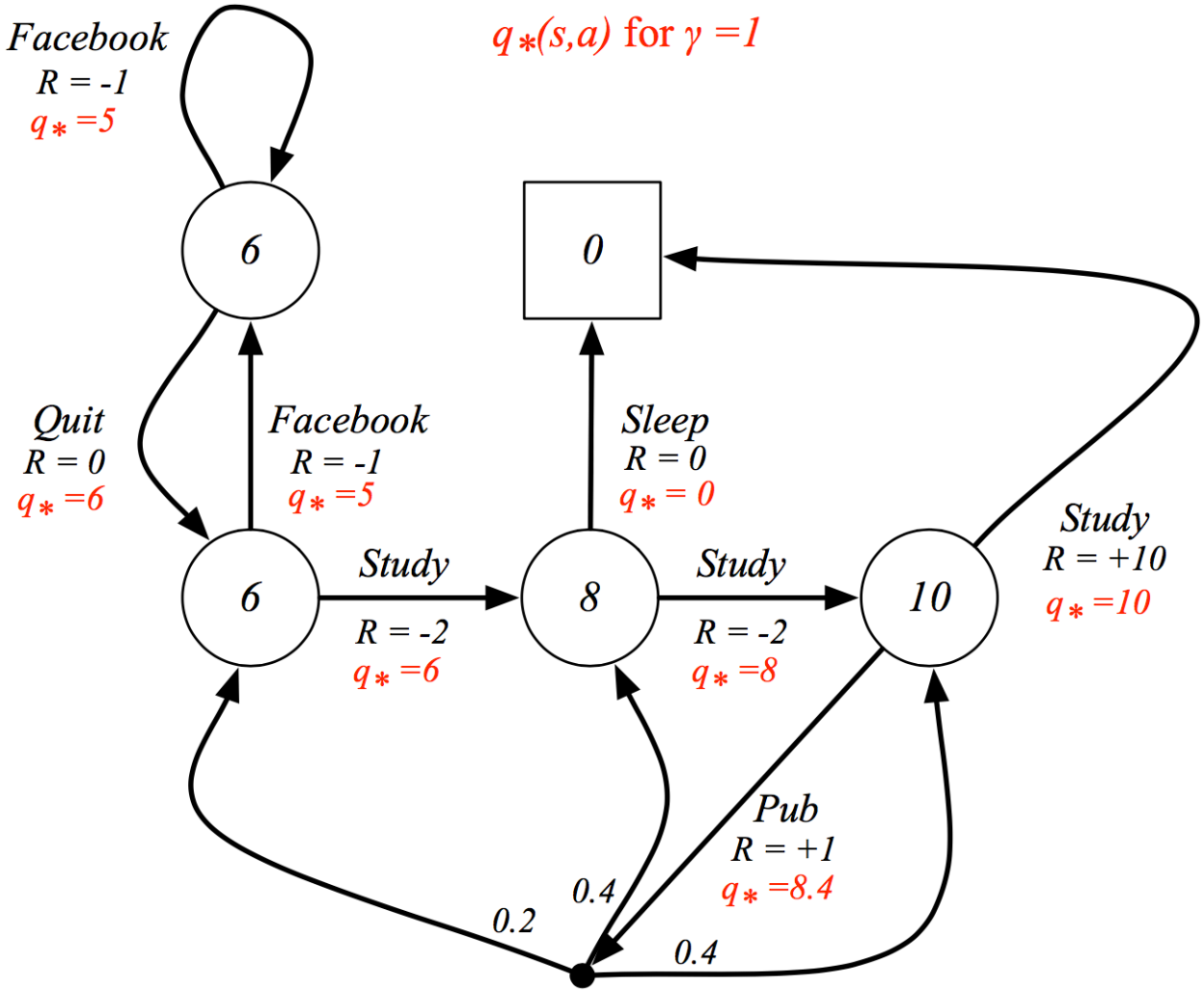
Definition 2: The optimal action-value function $q_*(s,a)$ is the maximum action-value function over all policies.

$$q_*(s, a) = \max q_\pi(s, a)$$

Example: Optimal AV-Function for Student MDP



Example: Optimal AV-Function for Student MCP



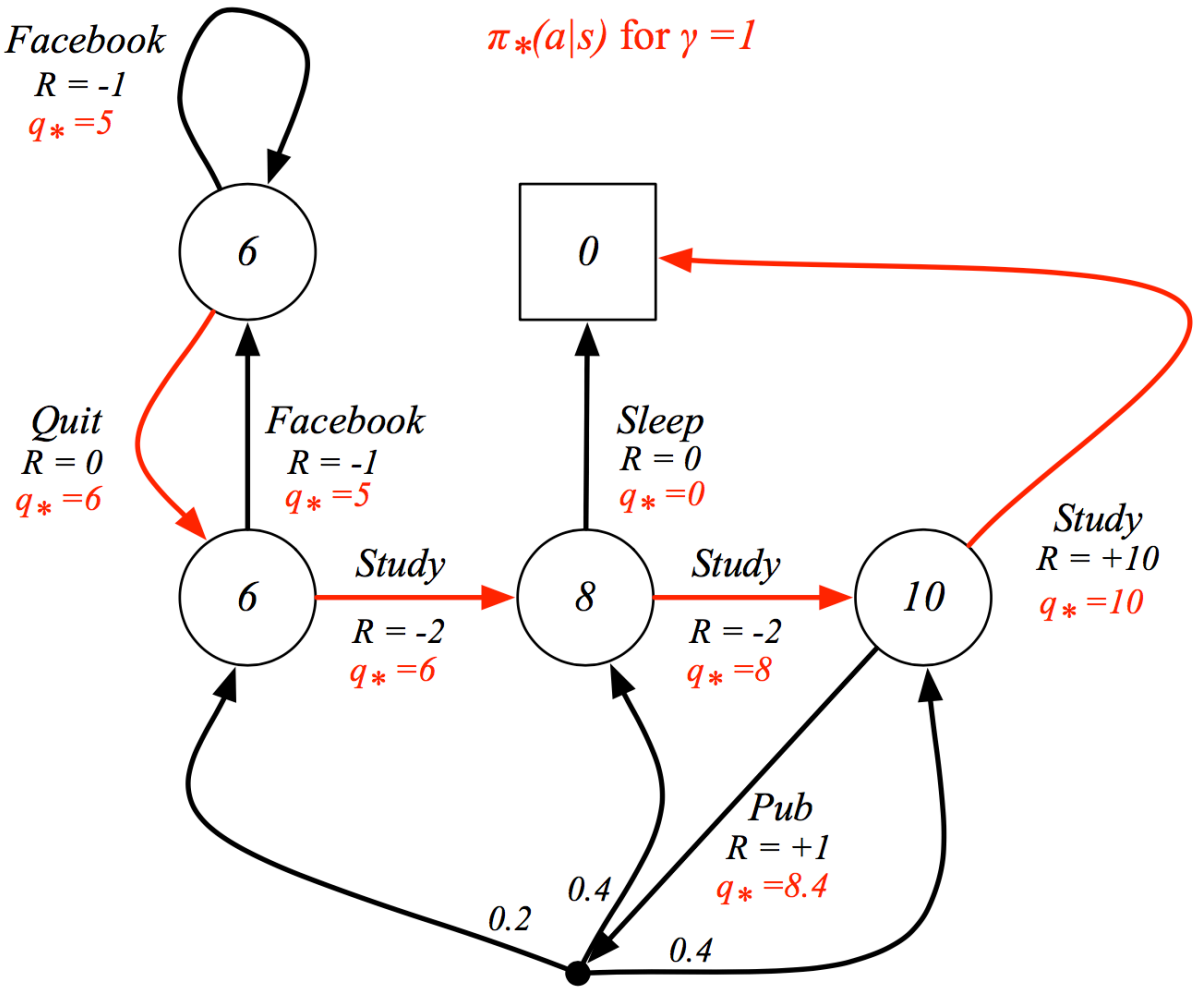
Finding the Optimal Policy.

- An optimal policy can be found by maximizing over $q_*(s,a)$,

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \underset{a \in \mathcal{A}}{\text{argmax}}\ q_*(s,a) \\ 0 & \textit{otherwise} \end{cases}$$

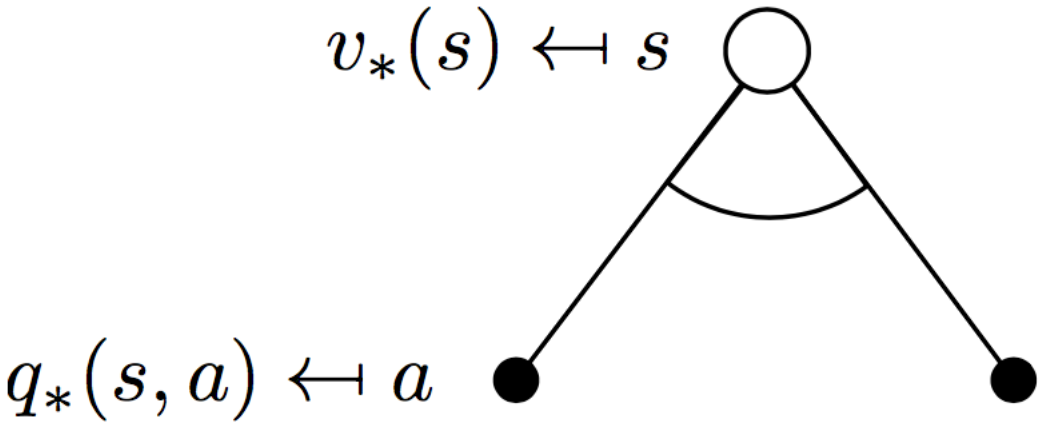
- There is always a deterministic optimal policy for any MDP.
- If we know $q_*(s,a)$, we immediately have the optimal policy.

Optimal Policy for Student MDP



The Bellmann Optimality Equation for V.

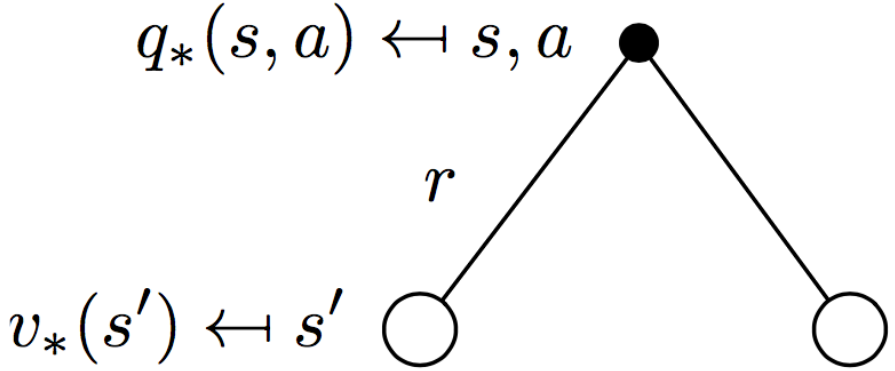
AKA **The Bellman Equation!**



$$v_*(s) = \max_a q_*(s,a)$$

Instead of taking the sum we take the max!

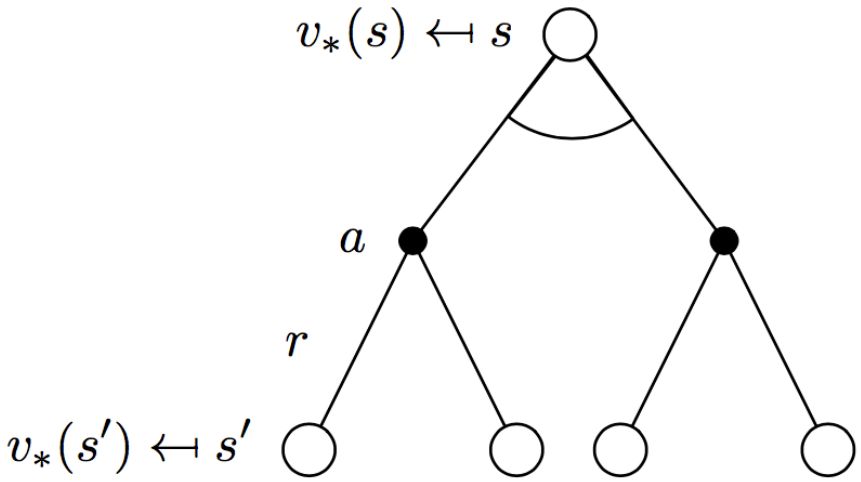
The Bellmann Optimality Equation for Q.



$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

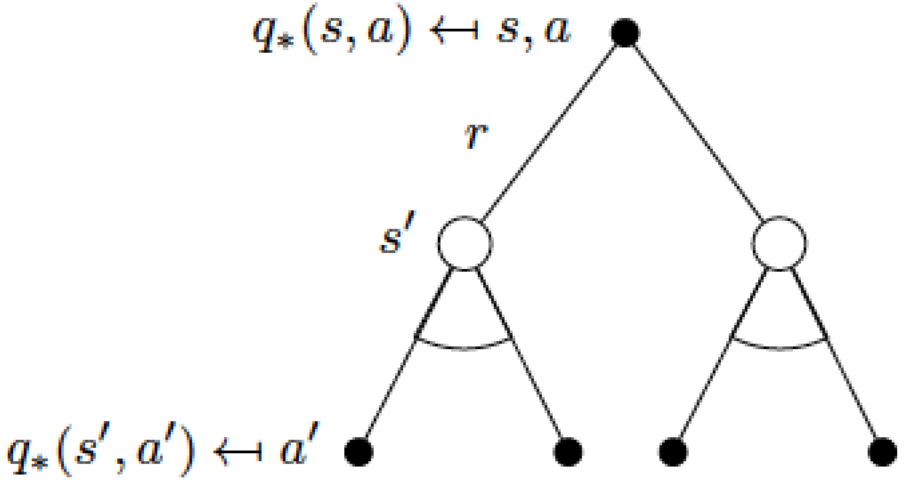
Here we sum, because we can't choose!

The Bellmann Optimality Equation for v_π



$$v_*(s) = \max_a \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

The Bellmann Optimality Equation for q_π

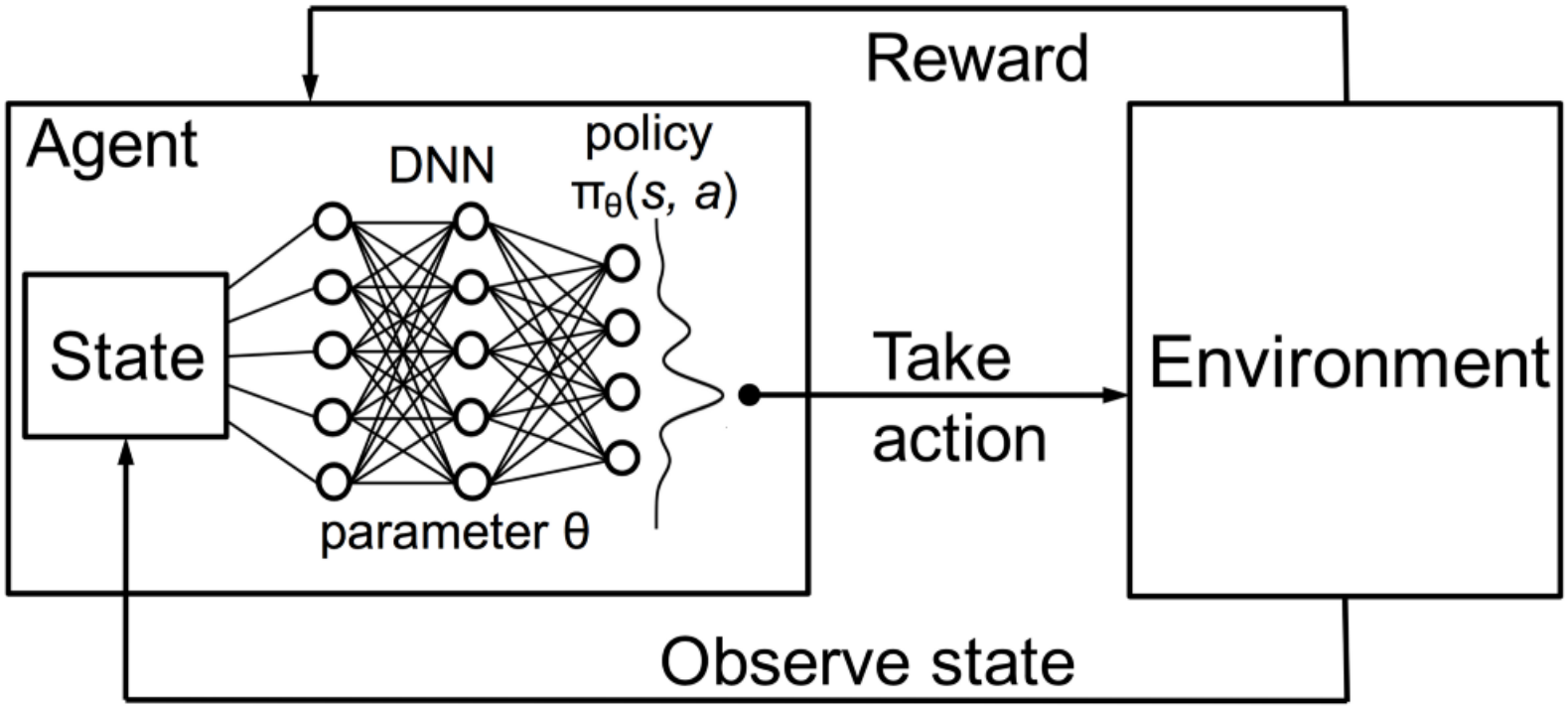


$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \max_{a'} q_*(s',a')$$

How to solve the Bellmann Optimality Equation?

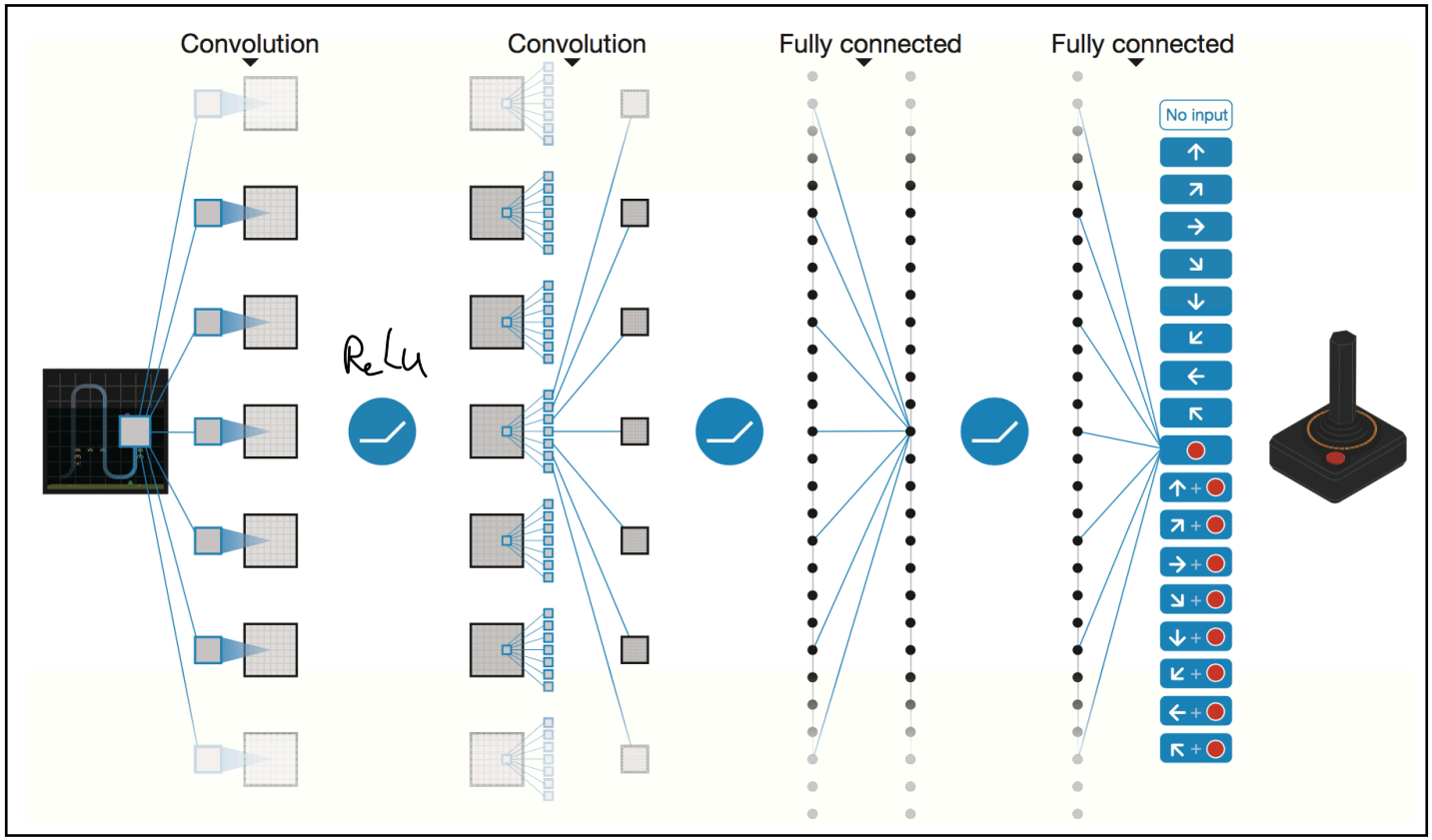
- The Bellman Optimality Equation is non linear.
- No closed form solution (in general).
- Many iterative solution methods.
  - Iterative Policy evaluation.
  - Iterative Value evaluation.
  - Q-learning

Deep Reinforcement Q-Learning.



$$L = \frac{1}{2}[\underbrace{r + max_{a'}Q(s', a')}_{\text{target}} - \underbrace{Q(s, a)}_{\text{prediction}}]^2$$

Deep Reinforcement Q-Learning.



Deep Reinforcement Q-Learning.

They used a Deep NN to approximate the optimal action-value function…

$$Q^*(s,a) = \max_{\pi} \mathbb{E}\left[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots \,|\, s_t = s,\, a_t = a,\, \pi\right],$$

With the following loss function to update the Q-learning rule:

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \mathrm{U}(D)}\left[\left(r + \gamma \max_{a'} Q(s',a';\theta_i^-) - Q(s,a;\theta_i)\right)^2\right]$$

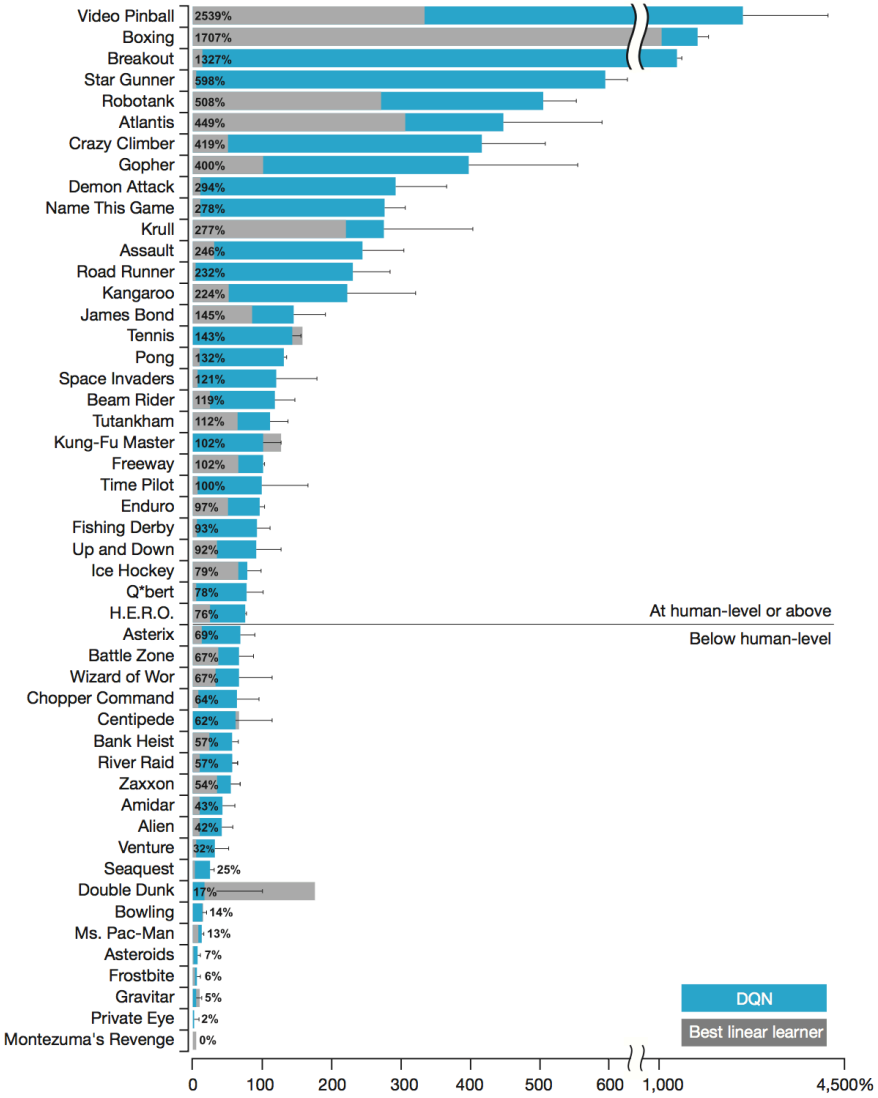
LETTER

doi:10.1038/nature14236

Human-level control through deep reinforcement learning

Volodymyr Mnih[1]*, Koray Kavukcuoglu[1]*, David Silver[1]*, Andrei A. Rusu[1], Joel Veness[1], Marc G. Bellemare[1], Alex Graves[1], Martin Riedmiller[1], Andreas K. Fidjeland[1], Georg Ostrovski[1], Stig Petersen[1], Charles Beattie[1], Amir Sadik[1], Ioannis Antonoglou[1], Helen King[1], Dharshan Kumaran[1], Daan Wierstra[1], Shane Legg[1] & Demis Hassabis[1]

Deep Reinforcement Q-Learning.



Montezuma's R

Deep Reinforcement Q-Learning.

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim \mathrm{U}(D)}\left[\left(r + \gamma \max_{a'} Q(s',a';\theta_i^-) - Q(s,a;\theta_i)\right)^2\right]$$

On-policy methods estimate the value of a policy while using it for control.
In **off-policy methods**, the policy used to generate behaviour, called the behaviour policy, may be unrelated to the policy that is evaluated and improved, called the estimation policy.

From the Sutton book: "The on-policy approach in the preceding section is actually a compromise—it learns action values not for the optimal policy, but for a near-optimal policy that still explores. A more straightforward approach is to use two policies, one that is learned about and that becomes the optimal policy, and one that is more exploratory and is used to generate behavior. The policy being learned about is called the target policy, and the policy used to generate behavior is called the behavior policy. In this case we say that learning is from data "o↵" the target policy, and the overall process is termed o↵-policy learning."

Deep Reinforcement Learning at Deep Mind

AlphaStar: Mastering the Real-Time Strategy Game StarCraft II

Games have been used for decades as an important way to test and evaluate the performance of artificial intelligence systems. As capabilities have increased, the research community has sought games with increasing complexity that capture different elements of intelligence required to solve scientific and real-world problems. In recent years, StarCraft, considered to be one of the most challenging Real-Time Strategy (RTS) games and one of the longest-played esports of all time, has emerged by consensus as a "grand challenge" for AI research.



Download 11 replays

deepmind.com/blog/alphastar-mastering-real-time-strategy-game-starcraft-ii/

deepmind.com/blog/article/AlphaStar-Grandmaster-level-in-StarCraft-II-using-multi-agent-reinforcement-learning

RL - What did we learn today?

- Basic components of RL
- The role of Dopamine in the Brain to predict Rewards
- Formalisation of the RL process (MDP)
- Policy and Value based RL
- Q-Learning
- Rescorla Wagner and Temporal Difference Rule
- The Bellmann (Expectation) Equation

Thank you for your Attention!

